BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO CUỐI KỲ

MÔN: PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

ĐỀ TÀI:

SMARTBIN — ỨNG DỤNG QUẢN LÝ HỆ THỐNG THÙNG RÁC THÔNG MINH

GVHD: Huỳnh Hoàng Hà

Họ và tên	MSSV
Võ Ngọc Mến	23119167
Đỗ Xuân Thanh	23119202
Lê Thành Nhân	24119162

TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN 1
1.1. Đặt vấn đề 1
1.2. Mục tiêu của đề tài 1
1.3. Đối tượng và phạm vi nghiên cứu 2
1.4. Phương pháp thực hiện 2
1.5. Kết quả dự kiến 3
1.6. Bố cục báo cáo 3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 4
2.1. Phân loại chất thải rắn sinh hoạt tại nguồn 4
2.2. React Native và Expo 4
2.3. Quản lý trạng thái và dữ liệu phía ứng dụng 4
2.4. Supabase 4
2.5. Kiến trúc "ghi cục bộ trước" (local-first) và đồng bộ ngoại tuyến 5
2.6. Bluetooth Low Energy và vi điều khiển ESP32 5
2.7. Học sâu cho phân loại ảnh và mô hình thị giác – ngôn ngữ 5
2.8. TensorFlow Lite và lượng tử hoá 6
2.9. Bài toán lộ trình và dự báo 6
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 7
3.1. Phân tích yêu cầu 7
3.1.1. Tác nhân 7
3.1.2. Yêu cầu chức năng 7
3.1.3. Yêu cầu phi chức năng 8
3.2. Kiến trúc hệ thống 9
3.3. Thiết kế cơ sở dữ liệu 10
3.4. Thiết kế xử lý 11
3.4.1. Luồng bỏ rác 11
3.4.2. Nhận diện rác trực tiếp bằng camera 15
3.4.3. Đồng bộ ngoại tuyến 20

3.4.4. Tự động hoá phía máy chủ 24
3.4.5. Lộ trình thu gom 30
3.5. Thiết kế phần cứng và giao thức BLE 34
3.6. Thiết kế giao diện và điều hướng 38
CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ 39
4.1. Môi trường thực hiện 39
4.2. Kết quả giao diện 39
4.3. Kết quả mô hình AI 41
4.4. Kết quả kiểm thử 42
4.4.1. Kiểm thử máy chủ 42
4.4.2. Kiểm thử logic ứng dụng 43
4.4.3. Kiểm thử trên điện thoại thật 43
4.5. Đánh giá mức độ đạt mục tiêu 44
CHƯƠNG 5: KẾT LUẬN 45
5.1. Kết quả đạt được 45
5.2. Hạn chế 45
5.3. Hướng phát triển 45
TÀI LIỆU THAM KHẢO 47
PHỤ LỤC: TÀI LIỆU BÀN GIAO 49

CHƯƠNG 1: TỔNG QUAN

1.1. Đặt vấn đề

Rác thải sinh hoạt là một trong những vấn đề môi trường lớn của các đô thị Việt Nam. Theo Luật Bảo vệ môi trường 2020, từ ngày 31/12/2024 các hộ gia đình, cá nhân bắt buộc phải phân loại chất thải rắn sinh hoạt tại nguồn thành ba nhóm: chất thải có khả năng tái sử dụng, tái chế; chất thải thực phẩm (hữu cơ); và chất thải rắn sinh hoạt khác. Tuy nhiên, trên thực tế người dân vẫn thường nhầm lẫn khi phân loại, còn việc thu gom chủ yếu theo lịch cố định: có thùng đầy tràn chưa được thu gom trong khi có thùng còn trống vẫn được ghé qua, gây lãng phí nhân lực và mất vệ sinh.

Sự phát triển của điện thoại thông minh, Internet vạn vật (IoT) và trí tuệ nhân tạo (AI) chạy trực tiếp trên thiết bị mở ra hướng giải quyết: thùng rác có thể tự mở đúng ngăn khi nhận ra loại rác, tự báo mức đầy, và hệ thống có thể tự lên lịch, tự phân công nhân viên thu gom dựa trên dữ liệu thực tế thay vì lịch cố định.

Xuất phát từ thực tế đó, nhóm chọn đề tài "SmartBin — Ứng dụng quản lý hệ thống thùng rác thông minh", xây dựng một ứng dụng di động kết nối người dân, nhân viên thu gom và người quản lý trên cùng một nền tảng dữ liệu thời gian thực.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát: xây dựng ứng dụng di động hoàn chỉnh hỗ trợ phân loại rác tại nguồn và quản lý thu gom rác một cách tự động. Các mục tiêu cụ thể:

- Hỗ trợ người dân phân loại đúng 3 nhóm rác (hữu cơ, vô cơ, tái chế): chọn tay, tra cứu, hoặc để AI nhận diện qua camera và tự mở đúng ngăn thùng.
- Khuyến khích phân loại bằng điểm thưởng, huy hiệu, thống kê tác động môi trường (khối lượng rác, lượng CO_2 giảm được).
- Giúp nhân viên thu gom nhận việc, xem lộ trình tối ưu, chụp ảnh nghiệm thu và đánh giá chất lượng phân loại.
- Giúp người quản lý giám sát mức đầy các thùng trên bản đồ theo thời gian thực, điều phối nhân viên, lên lịch ca trực.
- Tự động hoá vận hành: tự lên lịch khi thùng sắp đầy, tự giao việc, tự nhắc việc trễ, tự phát hiện thiết bị mất kết nối.
- Ứng dụng vẫn hoạt động khi mất mạng (ghi cục bộ, đồng bộ lại khi có mạng).

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng sử dụng gồm bốn nhóm:

Bảng 1.1: Đối tượng sử dụng và chức năng

Đối tượng	Chức năng chính
Người dân (kiosk cộng đồng)	Bỏ rác tại thùng công cộng, không cần tài khoản; AI tự nhận diện và mở ngăn
Hộ gia đình	Thùng rác riêng; tích điểm, lịch sử, thống kê, huy hiệu, lịch thu gom
Nhân viên thu gom	Nhận việc, lộ trình tối ưu, nghiệm thu bằng ảnh, chấm chất lượng phân loại
Người quản lý	Giám sát thùng, bản đồ, điều phối, lịch ca, cấu hình tự động hoá

Phạm vi:

- Ứng dụng di động Android (React Native / Expo), máy chủ Supabase (PostgreSQL, xác thực, lưu trữ ảnh, realtime).
- Phân loại theo 3 nhóm rác đúng quy định: hữu cơ, vô cơ, tái chế.
- Phần cứng ở mức mô hình: một thùng rác 3 ngăn điều khiển bằng ESP32 + servo qua Bluetooth Low Energy (BLE); có chế độ giả lập để demo khi không có phần cứng.
- Mô hình AI sử dụng mô hình đã công bố (MobileCLIP-S0, CVPR 2024), chạy trên điện thoại; không huấn luyện mô hình mới.

1.4. Phương pháp thực hiện

- **Khảo sát:** tìm hiểu quy định phân loại rác tại nguồn, các nghiên cứu về phân loại rác bằng học sâu và mô hình thị giác – ngôn ngữ (CLIP, MobileCLIP).
- **Thiết kế:** kiến trúc tách lớp hạ tầng (core) và nghiệp vụ (features); mô hình dữ liệu PostgreSQL với phân quyền theo vai trò (Row Level Security); thiết kế "ghi cục bộ trước, đồng bộ sau" (local-first) với SQLite và hàng đợi đồng bộ.
- **Hiện thực:** React Native + Expo Router, Zustand, TanStack Query; Supabase (trigger, pg_cron, Realtime); TensorFlow Lite + VisionCamera cho AI; BLE cho phần cứng.
- **Kiểm thử:** kiểm thử tự động phía máy chủ (44 phép kiểm tra trigger, phân quyền, tự động hoá), kiểm thử logic ứng dụng (25 phép), đánh giá AI trên 2.749 ảnh, kiểm thử thủ công trên điện thoại thật.

- **Quản lý dự án:** mã nguồn trên GitHub, báo cáo tiến độ hằng tuần, bảng phân công công việc.

1.5. Kết quả dự kiến

- Ứng dụng Android hoàn chỉnh cho 4 nhóm người dùng, chạy trên dữ liệu thật.
- Chức năng AI nhận diện rác trên thiết bị với độ chính xác trên 90%, tự mở đúng ngăn thùng.
- Hệ thống thu gom tự vận hành: tự lên lịch, tự phân công, tự cảnh báo.
- Mô hình thùng rác thông minh ESP32 kết nối với ứng dụng.
- Cuốn báo cáo, mã nguồn và kịch bản demo.

1.6. Bố cục báo cáo

- **Chương 1 – Tổng quan:** đặt vấn đề, mục tiêu, đối tượng, phạm vi, phương pháp.
- **Chương 2 – Cơ sở lý thuyết:** các công nghệ và mô hình được sử dụng.
- **Chương 3 – Phân tích và thiết kế hệ thống:** yêu cầu, kiến trúc, cơ sở dữ liệu, thiết kế giao diện và các luồng xử lý.
- **Chương 4 – Phân tích kết quả:** kết quả hiện thực, kiểm thử, đánh giá AI.
- **Chương 5 – Kết luận:** kết quả đạt được, hạn chế, hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Phân loại chất thải rắn sinh hoạt tại nguồn

Luật Bảo vệ môi trường số 72/2020/QH14 (Điều 75) và Nghị định 08/2022/NĐ-CP quy định chất thải rắn sinh hoạt phải được phân loại tại nguồn thành ba nhóm: (1) chất thải có khả năng tái sử dụng, tái chế; (2) chất thải thực phẩm; (3) chất thải rắn sinh hoạt khác. Ứng dụng SmartBin dùng đúng ba nhóm này với tên gọi quen thuộc: tái chế, hữu cơ và vô cơ. Việc phân loại đúng giúp tăng tỷ lệ tái chế, giảm lượng rác chôn lấp và giảm phát thải khí mê-tan từ rác hữu cơ.

2.2. React Native và Expo

React Native là framework của Meta cho phép viết ứng dụng di động bằng JavaScript/TypeScript với thư viện React; giao diện được hiển thị bằng thành phần gốc (native) của Android/iOS chứ không phải trang web. Phiên bản 0.76 dùng kiến trúc mới (Fabric, TurboModules) và máy ảo JavaScript Hermes được tối ưu cho di động.

Expo (SDK 52) cung cấp bộ công cụ build, các module dựng sẵn (camera, SQLite, thông báo, vị trí, lưu trữ bảo mật) và Expo Router — điều hướng theo tệp: mỗi tệp trong thư mục app/ là một màn hình, thư mục có ngoặc như (admin) là một nhóm màn hình dùng chung một layout (ví dụ thanh tab dưới). Vì app dùng module gốc (TFLite, VisionCamera, BLE) nên phải dùng development build / bản release thay vì Expo Go.

2.3. Quản lý trạng thái và dữ liệu phía ứng dụng

- **Zustand:** thư viện quản lý trạng thái toàn cục gọn nhẹ (hook create), dùng cho phiên đăng nhập, cấu hình kiosk, thời điểm cập nhật realtime.
- **TanStack Query:** quản lý dữ liệu lấy từ máy chủ: bộ nhớ đệm theo khoá (queryKey), tự tải lại, làm mới khi dữ liệu thay đổi (invalidateQueries). Các màn hình dùng chung một khoá sẽ dùng chung một bản dữ liệu.
- **SQLite (expo-sqlite):** cơ sở dữ liệu cục bộ trên điện thoại, lưu sự kiện bỏ rác và hàng đợi đồng bộ.

2.4. Supabase

Supabase là nền tảng backend mã nguồn mở xây dựng trên PostgreSQL, cung cấp:

- **PostgreSQL:** hệ quản trị CSDL quan hệ; hỗ trợ hàm PL/pgSQL, trigger (tự chạy khi INSERT/UPDATE), ràng buộc CHECK.

- **Auth:** đăng nhập bằng email/mật khẩu, cấp JSON Web Token (JWT); trong SQL, auth.uid() trả về người đang gọi.
- **Row Level Security (RLS):** chính sách phân quyền theo từng dòng dữ liệu. Ví dụ nhân viên chỉ đọc được việc giao cho mình; kiosk (vai trò anon) chỉ được ghi sự kiện không gắn người dùng. Phân quyền nằm ở CSDL nên không thể vượt qua từ app.
- **Realtime:** đọc nhật ký thay đổi của PostgreSQL (logical replication) và đẩy về app qua WebSocket; RLS vẫn áp dụng.
- **pg_cron:** lập lịch chạy câu lệnh SQL định kỳ (cú pháp cron) ngay trong CSDL.
- **Storage:** lưu tệp (ảnh nghiệm thu) trong bucket riêng tư, truy cập qua đường dẫn có chữ ký, có thời hạn.

2.5. Kiến trúc "ghi cục bộ trước" (local-first) và đồng bộ ngoại tuyến

Thùng rác đặt ngoài trời có thể mất mạng. Theo hướng local-first, mọi thao tác ghi được lưu ngay vào SQLite trên máy rồi đưa vào một hàng đợi; một bộ máy đồng bộ gửi dần hàng đợi lên máy chủ khi có mạng. Ba kỹ thuật chính:

- **Khoá chống trùng (idempotency):** mỗi sự kiện có local_id (UUID sinh trên máy, cột UNIQUE trên máy chủ), gửi lại nhiều lần cũng không tạo bản ghi trùng.
- **Thử lại có giới hạn:** lỗi mạng thì dừng và giữ nguyên hàng đợi; lỗi từ máy chủ thì tính một lần thất bại, tối đa 5 lần để một thao tác hỏng không chặn các thao tác phía sau.
- **Giải quyết xung đột:** last-write-wins theo thời điểm tạo — phù hợp vì dữ liệu sự kiện chủ yếu chỉ ghi thêm.

2.6. Bluetooth Low Energy và vi điều khiển ESP32

BLE là chuẩn Bluetooth tiết kiệm năng lượng. Thiết bị ngoại vi (ESP32) đóng vai trò GATT server, cung cấp các service gồm nhiều characteristic; điện thoại (GATT client) quét, kết nối, ghi và đọc characteristic. ESP32 là vi điều khiển Wi-Fi/BLE giá rẻ; servo được điều khiển bằng xung PWM: servo 180° quay nắp đến góc mong muốn, còn servo quay liên tục 360° quay theo chiều và tốc độ định bởi độ rộng xung, nên nắp được mở bằng cách quay trong một khoảng thời gian.

2.7. Học sâu cho phân loại ảnh và mô hình thị giác – ngôn ngữ

Các nghiên cứu phân loại rác thường huấn luyện mạng tích chập (CNN) trên tập ảnh có nhãn, ví dụ TrashNet [3] với 6 loại vật liệu. Hạn chế: cần dữ liệu lớn cho đúng các nhóm của bài toán, và muốn đổi nhóm phải huấn luyện lại.

CLIP [2] huấn luyện đồng thời bộ mã hoá ảnh và bộ mã hoá văn bản trên hàng trăm triệu cặp ảnh – chú thích bằng học tương phản (contrastive learning): ảnh và câu mô tả đúng của nó được kéo lại gần nhau trong cùng một không gian vector. Nhờ vậy có thể phân loại zero-shot: không cần huấn luyện thêm, chỉ cần so ảnh với các câu mô tả từng lớp.

Với vector ảnh v và vector câu mô tả t_i đã chuẩn hoá về độ dài 1, xác suất của câu i là:

$$p(i) = \exp(100 \cdot \cos(v, t_i)) \,/\, \Sigma_j \exp(100 \cdot \cos(v, t_j))$$

trong đó 100 là hệ số nhiệt độ (logit scale) học được của CLIP. Kỹ thuật prompt ensembling [2] dùng nhiều câu mô tả cho mỗi lớp (nhiều đồ vật × nhiều mẫu câu); điểm của một lớp là tổng xác suất các câu thuộc lớp đó.

MobileCLIP [1] (Apple, CVPR 2024) là họ mô hình CLIP thiết kế cho thiết bị di động, huấn luyện theo phương pháp "multi-modal reinforced training" để giữ độ chính xác cao với độ trễ thấp. Bản nhỏ nhất MobileCLIP-S0 có bộ mã hoá ảnh lai CNN–Transformer, đầu vào 256×256, đầu ra vector 512 chiều. Hướng zero-shot bằng mô hình thị giác – ngôn ngữ cũng đã được áp dụng cho phân loại rác nhựa trong [6].

2.8. TensorFlow Lite và lượng tử hoá

TensorFlow Lite (LiteRT) là môi trường chạy mô hình học sâu trên thiết bị. Delegate cho phép chuyển tính toán sang GPU (OpenCL/OpenGL) để tăng tốc. Lượng tử hoá giảm độ chính xác số của trọng số: FP16 (16 bit) giảm một nửa dung lượng; INT8 dynamic range lưu trọng số 8 bit và giải lượng tử khi chạy; lượng tử hoá nguyên hoàn toàn [5] tính toán hoàn toàn bằng số nguyên nên nhanh nhất trên phần cứng di động.

2.9. Bài toán lộ trình và dự báo

- **Khoảng cách haversine:** khoảng cách trên mặt cầu giữa hai toạ độ (vĩ độ, kinh độ), bán kính Trái Đất 6.371 km.
- **Bài toán người du lịch (TSP):** tìm thứ tự ghé thăm các điểm có tổng quãng đường nhỏ nhất — bài toán NP-khó. Với vài chục điểm, heuristic láng giềng gần nhất (luôn đi tới điểm gần nhất chưa ghé) kết hợp cải thiện 2-opt [7] (đảo ngược một đoạn đường nếu làm tổng quãng đường ngắn lại) cho lời giải gần tối ưu trong thời gian rất ngắn.
- **Dự báo tuyến tính:** tốc độ đầy của một ngăn = số lượt bỏ rác trong cửa sổ quan sát × mức đầy mỗi lượt; thời gian còn lại đến ngưỡng = (ngưỡng − mức hiện tại) / tốc độ.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Tác nhân

Bảng 3.1: Các tác nhân của hệ thống

Tác nhân	Mô tả
Người dân (kiosk)	Bỏ rác ở thùng công cộng qua màn hình kiosk gắn trên thùng; không cần tài khoản.
Hộ gia đình	Sở hữu một thùng rác riêng; có tài khoản, tích điểm.
Nhân viên thu gom	Nhận và thực hiện việc thu gom, nghiệm thu bằng ảnh.
Quản lý	Giám sát toàn hệ thống, điều phối nhân viên, cấu hình tự động hoá.
Hệ thống (máy chủ)	Tác nhân tự động: trigger và lịch pg_cron tự lên lịch, giao việc, cảnh báo.
Thùng rác ESP32	Thiết bị mở ngăn theo lệnh BLE.

3.1.2. Yêu cầu chức năng

Bảng 3.2: Yêu cầu chức năng

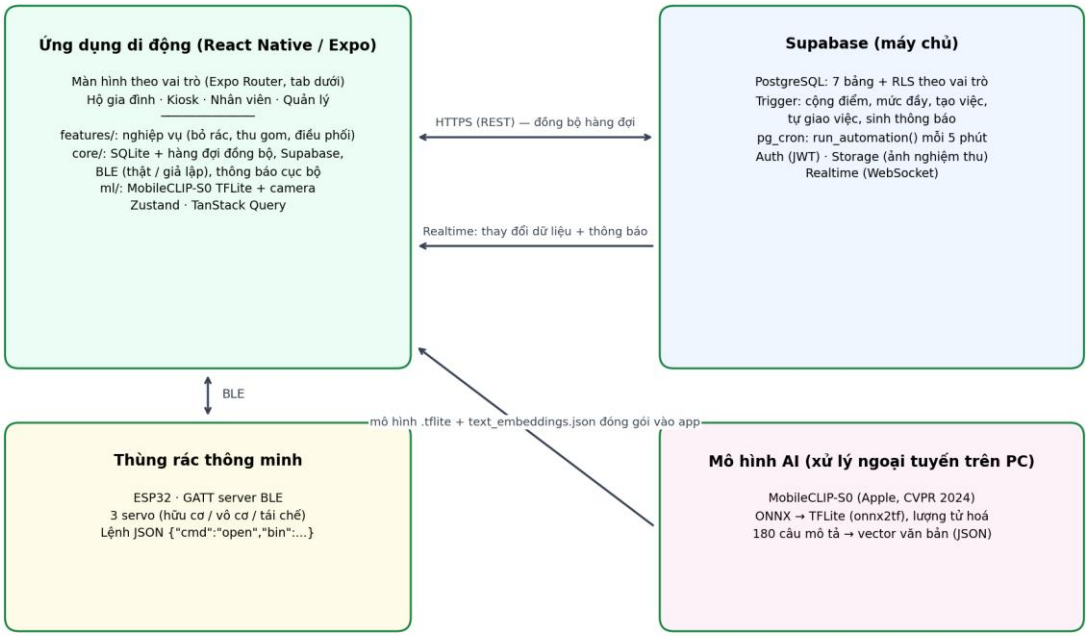
Mã	Tác nhân	Chức năng
F01	Tất cả	Chọn chế độ Cộng đồng / Hộ gia đình; đăng nhập đúng vai trò của chế độ
F02	Kiosk, Hộ GĐ	Bỏ rác: chọn loại, tra cứu, chụp ảnh AI, hoặc camera AI tự nhận diện → mở đúng ngăn
F03	Kiosk, Hộ GĐ	Hoạt động khi mất mạng, tự đồng bộ khi có mạng
F04	Hộ GĐ	Điểm thưởng, lịch sử, thống kê, tác động môi trường, huy hiệu, chuỗi ngày
F05	Hộ GĐ	Xem lịch thu gom kế tiếp; nhận thông báo lên lịch, đã thu gom, đánh giá phân loại
F06	Nhân viên	Danh sách việc, nhận việc, lộ trình tối ưu, chỉ đường
F07	Nhân viên	Hoàn tất việc kèm ảnh nghiệm thu; chấm chất lượng phân loại thùng hộ gia đình

F08	Quản lý	Bản đồ và mức đầy từng ngăn theo thời gian thực; dự báo thời điểm đầy
F09	Quản lý	Giao / đổi / huỷ giao việc, lên lịch ca trực, xem ảnh nghiệm thu
F10	Quản lý	Bật/tắt quy tắc tự động, chọn nhân viên đang trực, xem nhật ký tự động
F11	Hệ thống	Tự lên lịch khi ngăn ≥60%, tạo/nâng việc khẩn khi ≥80%, tự giao việc
F12	Hệ thống	Định kỳ: nhắc/leo thang việc trễ, dự báo theo tốc độ, phát hiện thùng mất kết nối
F13	Hệ thống	Gửi thông báo đúng người, hiển thị thông báo trên điện thoại

3.1.3. Yêu cầu phi chức năng

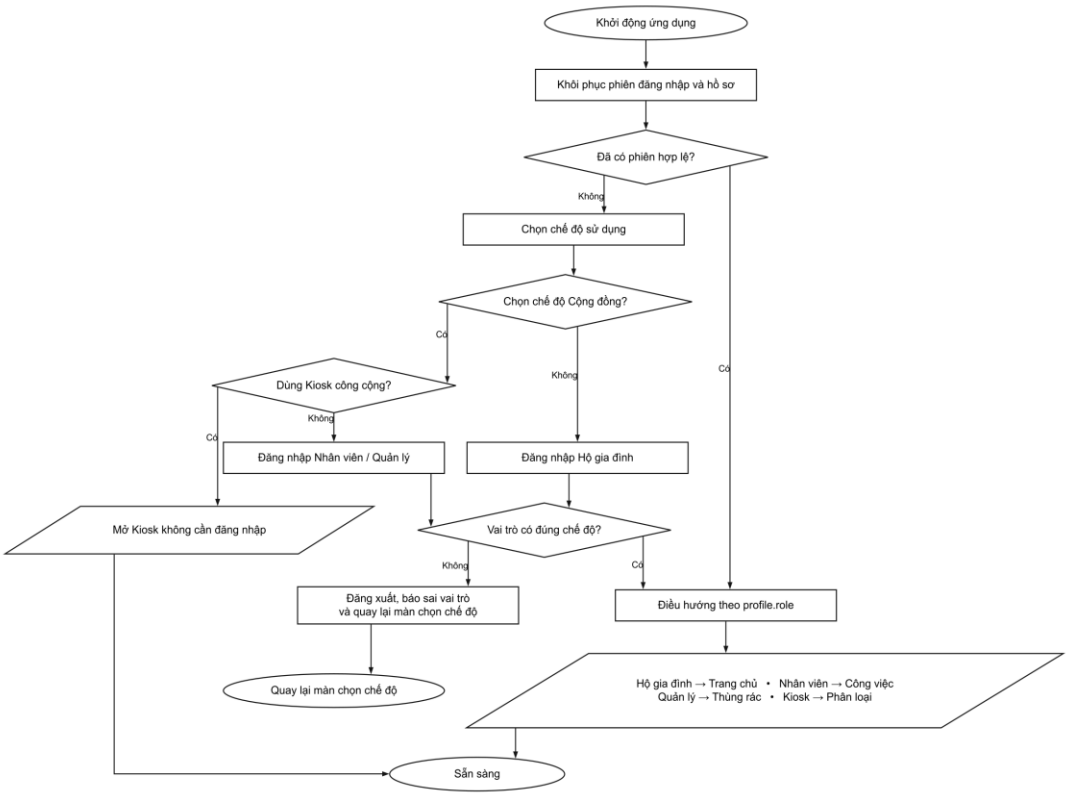
- **Khả dụng ngoại tuyến:** thao tác bỏ rác không phụ thuộc mạng.
- **Bảo mật:** phân quyền ở CSDL (RLS); điểm thưởng chỉ do máy chủ cộng; ảnh nghiệm thu không công khai.
- **Hiệu năng:** AI chạy trên máy, không gửi ảnh ra ngoài; nhận diện trực tiếp dưới 1 giây/khung.
- **Thời gian thực:** thay đổi hiển thị trên các máy khác trong khoảng 1–2 giây.
- **Dễ bảo trì:** tách hạ tầng và nghiệp vụ; module AI độc lập; có bản giả lập phần cứng để demo.

3.2. Kiến trúc hệ thống



Hình 3.1: Kiến trúc tổng thể hệ thống SmartBin

Ứng dụng chia thành các lớp: app/ (màn hình theo vai trò), features/ (nghiệp vụ: bỏ rác, thu gom, điều phối, thông báo, thống kê), core/ (hạ tầng: Supabase, SQLite, đồng bộ, BLE, thông báo — không biết gì về rác), ml/ (mô hình AI, nơi duy nhất dùng TFLite và camera) và shared/ (giao diện, kiểu dữ liệu dùng chung). Các feature chỉ nhận kết quả dạng {loại rác, độ tin cậy}, không phân biệt do người chọn hay AI suy ra.



Hình 3.2: Lưu đồ khởi động, đăng nhập và điều hướng theo vai trò

3.3. Thiết kế cơ sở dữ liệu

Bảng 3.3: Các bảng dữ liệu

Bảng	**Cột chính**	**Ý nghĩa**
profiles	id, full_name, role, points, on_duty	Người dùng; vai trò household/collector/admin; nhân viên đang trực
devices	id, code, name, area, latitude, longitude, is_online, last_seen_at, owner_id	Thùng rác; owner_id ≠ null là thùng hộ gia đình
bins	id, device_id, waste_type, fill_level	Ba ngăn của mỗi thùng, mức đầy 0..1
sort_events	id, local_id (unique), device_id, user_id, waste_type, source (manual/ai), confidence	Mỗi lượt bỏ rác

collection_tasks	id, device_id, assignee_id, status, priority, shift, scheduled_date, origin, auto_assigned, proof_photo_url, sorting_quality, escalated_at	Việc thu gom; origin cho biết việc do quản lý hay hệ thống tạo
notifications	id, recipient_id (null = mọi quản lý), kind, title, body, device_id, task_id, read_at	Thông báo / nhật ký tự động
automation_settings	auto_dispatch, predictive_schedule, escalate_after_min, offline_after_min	Cấu hình tự động hoá (1 dòng)

Phân quyền (Row Level Security) tiêu biểu:

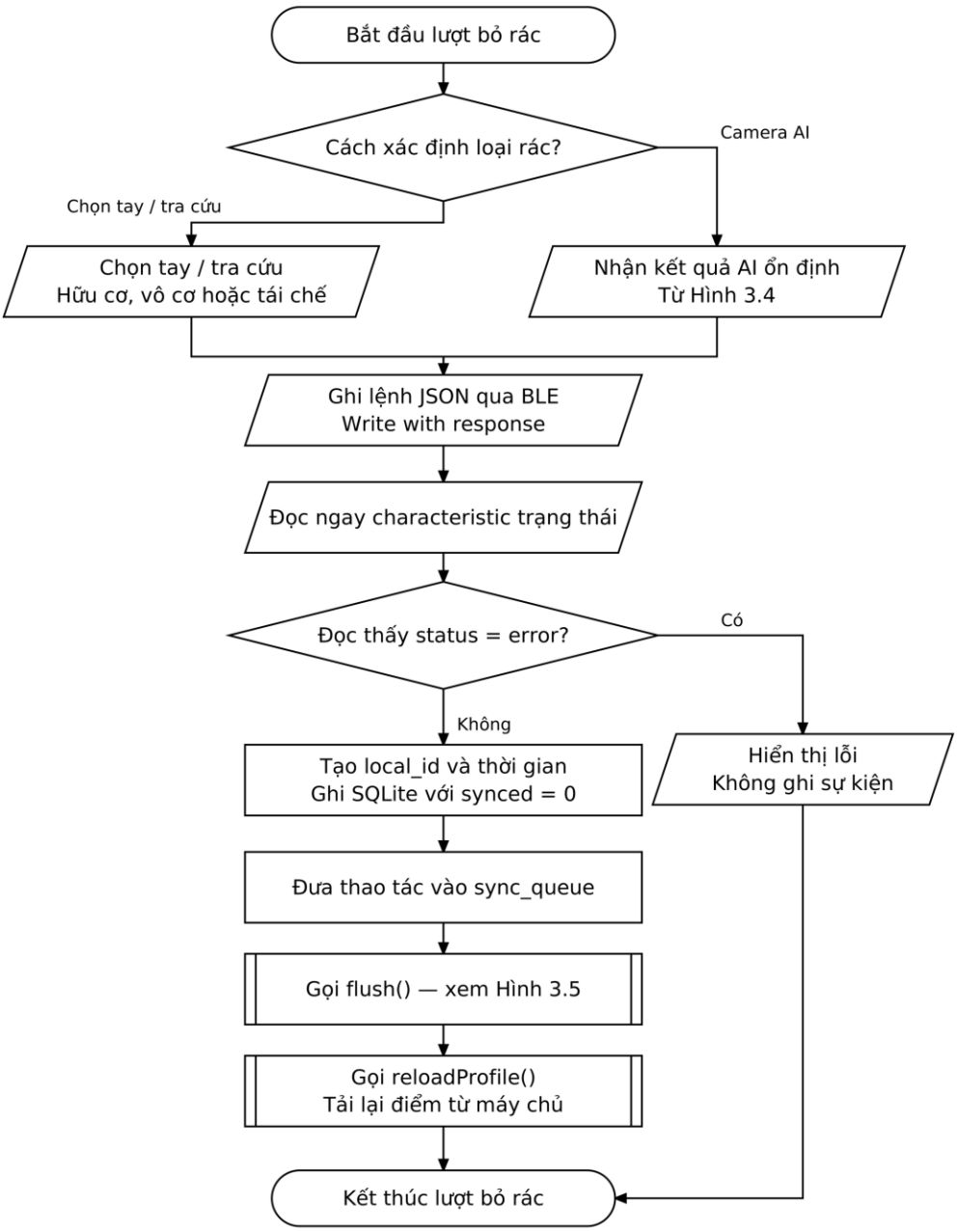
Bảng 3.4: Chính sách phân quyền tiêu biểu

Bảng	**Chính sách**
sort_events	Người đăng nhập chỉ ghi sự kiện của chính mình; kiosk (anon) chỉ ghi sự kiện user_id rỗng
collection_tasks	Nhân viên xem việc giao cho mình và việc chưa ai nhận; hộ gia đình xem việc của thùng nhà mình; quản lý xem tất cả
notifications	Mỗi người chỉ đọc thông báo gửi cho mình; nhật ký chung (recipient null) chỉ quản lý đọc được
automation_settings, bins	Chỉ quản lý được sửa

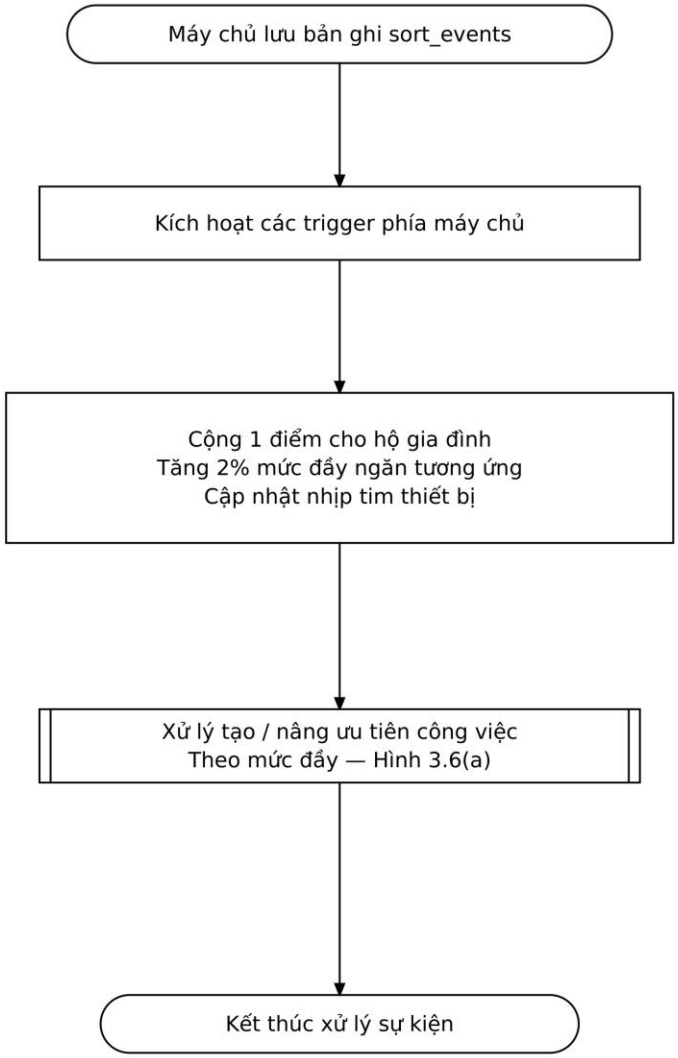
3.4. Thiết kế xử lý

3.4.1. Luồng bỏ rác

- Xác định loại rác (chọn tay / tra cứu / AI).
- Gửi lệnh mở ngăn tới thùng qua BLE (hoặc bản giả lập).
- Ghi sự kiện vào SQLite với local_id mới → đưa vào hàng đợi đồng bộ → thử gửi ngay nếu có mạng.
- Trên máy chủ, trigger khi có sort_events mới: cộng 1 điểm (hộ gia đình), tăng 2% mức đầy ngăn tương ứng, cập nhật nhịp tim thiết bị; nếu mức đầy vượt ngưỡng thì kích hoạt chuỗi tự động (3.4.4).



Hình 3.3(a): Lưu đồ phân loại và ghi nhận một lượt bỏ rác trên ứng dụng



Hình 3.3(b): Lưu đồ xử lý sự kiện bỏ rác phía máy chủ

Phân tích thuật toán Hình 3.3. Thuật toán ghi nhận một lượt bỏ rác (hàm sort() trong useSortAction.ts) có hai mục tiêu: chỉ ghi sự kiện khi thùng đã chấp nhận lệnh, và không làm mất lượt bỏ rác khi điện thoại mất mạng. Lưu đồ tách phần chạy trên ứng dụng (a) và phần chạy trên máy chủ sau khi sự kiện được đồng bộ (b).

Đầu vào và đầu ra

- **Đầu vào:** mã thùng (deviceId), loại rác (huu_co / vo_co / tai_che), nguồn nhãn (manual hoặc ai), độ tin cậy (chỉ có khi nguồn là AI) và mã người dùng (rỗng đối với kiosk cộng đồng).

- **Đầu ra:** giá trị true / false để giao diện chuyển màn hoặc báo lỗi; một bản ghi sort_events trong SQLite và một job trong sync_queue. Sau khi đồng bộ, máy chủ cập nhật điểm thưởng, mức đầy và nhịp tim thiết bị.

Diễn giải từng bước trên ứng dụng (Hình 3.3(a))

- **Bước 1 – Xác định loại rác:** hai nhánh "Chọn tay / tra cứu" và "Nhận kết quả AI ổn định" hội tụ về cùng một bước. Nhãn AI chỉ đến được đây sau khi đã qua ngưỡng tin cậy và kiểm tra độ ổn định (Hình 3.4), hoặc sau khi người dùng xác nhận kết quả chụp ảnh. Phần còn lại của thuật toán chỉ nhận cặp { loại, độ tin cậy }, không phụ thuộc nguồn gốc nhãn.
- **Bước 2 – Gửi lệnh:** chuỗi {"cmd":"open","bin":"<loại>"} được mã hoá base64 và ghi vào characteristic lệnh theo kiểu write with response, tức điện thoại chờ ESP32 xác nhận đã nhận gói tin. Nếu kết nối vừa rớt nhưng còn nhớ thiết bị, BinController thử kết nối lại một lần trước khi gửi.
- **Bước 3 – Đọc trạng thái:** đọc characteristic trạng thái và giải mã JSON. Có hai trường hợp đi vào nhánh lỗi: status = "error" (kèm thông điệp từ ESP32), hoặc dữ liệu không giải mã được ("Thiết bị trả về dữ liệu không hợp lệ").
- **Bước 4 – Tạo định danh:** local_id là UUID ngẫu nhiên sinh ngay trên máy, created_at là thời điểm hiện tại theo ISO 8601. Hai giá trị được tạo một lần rồi dùng chung cho SQLite, hàng đợi và máy chủ, nên cùng một lượt bỏ rác luôn có cùng định danh ở cả ba nơi.
- **Bước 5 – Ghi cục bộ rồi xếp hàng:** INSERT vào bảng sort_events với synced = 0, sau đó enqueue('sort_events', 'insert', payload). Ghi bảng trước, xếp hàng sau bảo đảm lịch sử trên máy luôn có lượt bỏ rác này, kể cả khi bước đồng bộ thất bại.
- **Bước 6 – Thử đồng bộ ngay:** gọi flush() (Hình 3.5). Hàm tự thoát nếu không có mạng hoặc đang có một lượt flush khác chạy, nên không làm chậm thao tác của người dùng.
- **Bước 7 – Tải lại hồ sơ:** reloadProfile() lấy điểm mới nhất từ máy chủ. Lệnh này không được chờ, nên giao diện chuyển sang màn tiếp theo ngay.

Diễn giải phía máy chủ (Hình 3.3(b))

- **Ba trigger AFTER INSERT** trên sort_events chạy cho từng dòng mới: add_points_on_sort cộng 1 điểm, nhưng chỉ khi user_id khác rỗng và người đó có vai trò household (lượt bỏ rác ở kiosk không cộng điểm cho ai); fill_bin_on_sort tăng mức đầy của ngăn cùng loại rác thêm 0,02, có chặn trên least(1, …) nên mức đầy không

vượt quá 100%; heartbeat_on_sort gọi device_heartbeat() để cập nhật last_seen_at và is_online của thùng.

- **Chuỗi trigger:** bước tăng mức đầy là một lệnh UPDATE trên bảng bins, nên kích hoạt tiếp trigger bins_fill_alert của Hình 3.6(a). Vì vậy một lượt bỏ rác có thể dẫn tới tạo việc, giao việc và gửi thông báo, tất cả trong cùng một giao dịch với lệnh INSERT ban đầu.

Đánh giá thuật toán

- **Tính đúng của điểm thưởng:** điểm chỉ do máy chủ cộng, ứng dụng không có quyền sửa cột points. Người dùng không thể tự tăng điểm bằng cách can thiệp dữ liệu trên máy.
- **Chống ghi trùng:** local_id là cột unique trên máy chủ, nên cùng một lượt bỏ rác dù được gửi lại nhiều lần cũng chỉ tạo một bản ghi và chỉ được cộng điểm một lần.
- **Chống bấm lặp:** cờ busy được bật trong suốt hàm sort(), giao diện khoá các nút chọn loại cho tới khi lượt hiện tại kết thúc.
- **Độ phức tạp:** mỗi lượt gồm một lần ghi và một lần đọc BLE, hai câu lệnh SQLite và tối đa một yêu cầu HTTP. Số bước không phụ thuộc vào số lượt đã có, tức độ phức tạp O(1).

Trường hợp biên và hạn chế

- **Mất mạng:** lượt bỏ rác vẫn được ghi nhận và hiển thị trong lịch sử, nhưng điểm chỉ được cộng sau khi đồng bộ thành công.
- **Mức kiểm tra của bước 3:** trạng thái được đọc ngay sau khi ghi lệnh, nên chỉ phản ánh các lỗi ESP32 phát hiện tức thời (lệnh sai, tên ngăn sai, hàng đợi đầy). Trạng thái ok được ESP32 ghi sau khi servo chạy xong (Hình 3.8(b)). Vì vậy "không có lỗi" ở bước này nghĩa là thùng đã nhận và xếp lệnh, chưa phải xác nhận nắp đã mở.

3.4.2. Nhận diện rác trực tiếp bằng camera

Quy trình mỗi khung hình (chạy 2 khung/giây trong frame processor của VisionCamera):

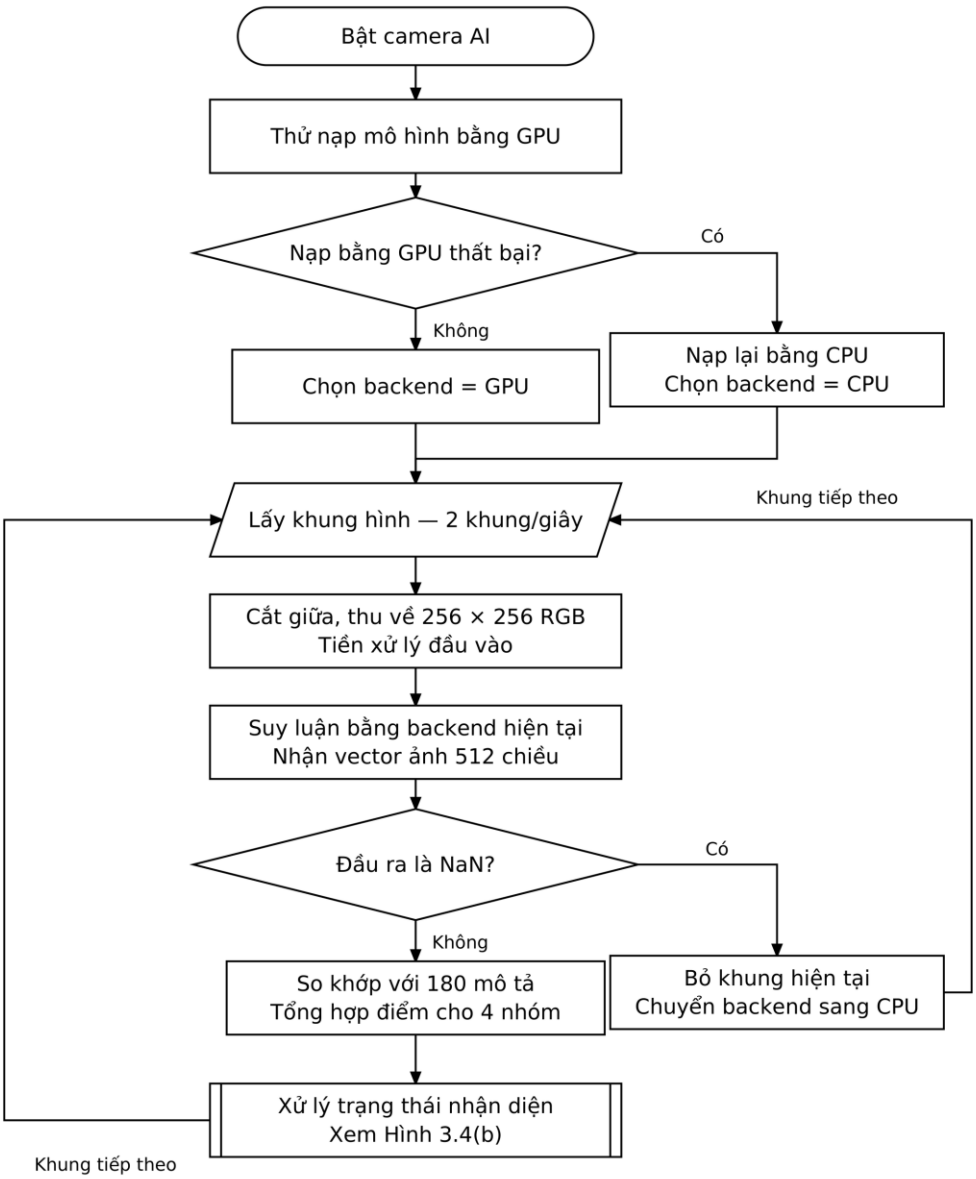
```
Khung camera → cắt giữa, thu về 256×256 RGB → MobileCLIP-S0 (TFLite, GPU) →
vector 512 chiều
→ cosine với 180 câu mô tả (4 nhóm: hữu cơ, vô cơ, tái chế, "không có rác") →
điểm từng nhóm
```

Để tránh mở nhầm, ứng dụng dùng một máy trạng thái:

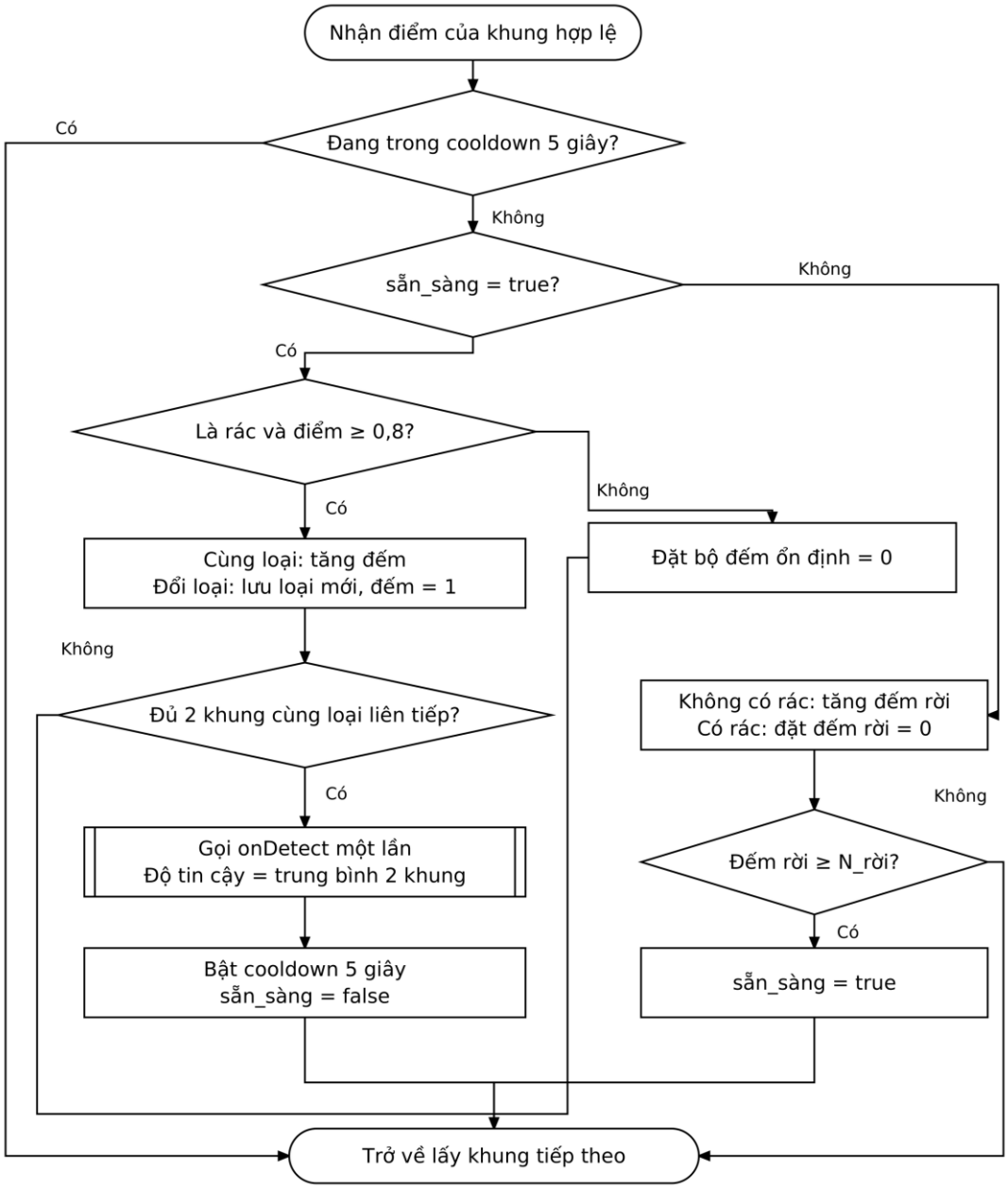
Bảng 3.5: Máy trạng thái nhận diện trực tiếp

Trạng thái	**Điều kiện chuyển**
Chờ (waiting)	Nhóm cao nhất là một loại rác với điểm ≥ 0,8 → Xác nhận
Xác nhận (detecting)	2 khung liên tiếp cùng loại → mở ngăn, gửi sự kiện → Nghỉ; khác loại → đếm lại từ 1 cho loại mới; không có rác → Chờ
Nghỉ (cooldown)	Sau 5 giây → Chờ vật rời khung
Chờ vật rời khung (clear)	2 khung liên tiếp không thấy rác → Chờ (vật nằm yên chỉ tính một lần)

Nhóm "không có rác" (mặt người, tay không, mặt bàn...) giúp camera không tự mở nắp khi chưa có rác. Delegate GPU được thử trước, lỗi hoặc ra giá trị NaN thì tự chuyển sang CPU.



Hình 3.4(a): Lưu đồ vòng lặp suy luận MobileCLIP-S0 trên từng khung hình



Hình 3.4(b): Lưu đồ máy trạng thái xác nhận kết quả nhận diện

Phân tích thuật toán Hình 3.4. Thuật toán nhận diện trực tiếp (useLiveWasteDetector.ts) biến luồng ảnh camera liên tục thành nhiều nhất một lần gọi onDetect cho mỗi vật rác được đưa vào. Hình 3.4(a) là vòng lặp xử lý từng khung hình; Hình 3.4(b) là máy trạng thái quyết định khi nào kết quả đủ tin cậy để mở nắp.

Tham số và biến trạng thái

- **Tham số:** tần số xử lý 2 khung/giây; ảnh đầu vào 256 × 256 RGB, giá trị thực trong [0, 1]; ngưỡng tin cậy 0,8; cần 2 khung liên tiếp cùng loại; cooldown 5 giây; cần 2 khung trống liên tiếp (N_rời = 2) để sẵn sàng lại.
- **Biến trạng thái:** backend đang dùng (GPU hoặc CPU); bộ đếm ổn định gồm loại rác, số khung và tổng độ tin cậy; thời điểm hết cooldown; cờ sẵn_sàng; bộ đếm rời đếm số khung trống liên tiếp.

Diễn giải vòng lặp khung hình (Hình 3.4(a))

- **Chọn backend:** trên Android, mô hình được nạp bằng GPU delegate trước. Nếu nạp lỗi (máy không hỗ trợ), mô hình được nạp lại bằng CPU.
- **Lấy và tiền xử lý khung:** frame processor của VisionCamera chạy trên luồng camera và giới hạn ở 2 khung/giây. Khung được cắt giữa, xoay theo hướng máy và thu về 256 × 256. Khung đến lúc mô hình còn bận sẽ bị camera bỏ qua, nên khung chờ xử lý không bị dồn lại.
- **Suy luận:** mô hình chạy đồng bộ (runSync) ngay trong frame processor, vì bộ đệm ảnh chỉ hợp lệ trong khung hiện tại. Đầu ra là vector ảnh 512 chiều, được chuyển sang luồng JavaScript để xử lý tiếp.
- **Kiểm tra NaN:** GPU thường tính ở FP16 nên có thể tràn số. Nếu đầu ra là NaN, khung hiện tại bị bỏ và backend chuyển hẳn sang CPU cho các khung sau trong phiên.
- **Phân loại zero-shot:** vector ảnh được chuẩn hoá về độ dài 1, rồi tính tích vô hướng với 180 vector mô tả đã tính sẵn bằng text encoder, nhân hệ số 100 và đưa qua softmax. Điểm của mỗi nhóm bằng tổng xác suất các câu mô tả thuộc nhóm đó. Kết quả dùng cho bước sau là nhóm cao nhất trên cả 4 nhóm (liveLabel) cùng điểm của nó (liveConfidence).
- **Lặp:** gọi máy trạng thái ở Hình 3.4(b), rồi quay lại lấy khung tiếp theo. Vòng lặp không có điểm kết thúc, chỉ dừng khi rời màn hình hoặc tắt camera.

Diễn giải máy trạng thái (Hình 3.4(b))

- **Kiểm tra 1 – đang cooldown:** trong 5 giây sau mỗi lần mở nắp, khung bị bỏ qua và bộ đếm ổn định đặt về 0. Màn hình bị tạm dừng cũng được xử lý giống hệt nhánh này.
- **Kiểm tra 2 – chưa sẵn sàng:** lúc này thuật toán chỉ đếm số khung trống liên tiếp; gặp khung có rác thì đếm lại từ 0. Đủ 2 khung trống thì đặt sẵn_sàng = true. Cờ này ban

đầu là false, nên nếu vừa bật camera mà đã có vật nằm trước ống kính, hệ thống cũng không mở nắp cho tới khi vật được lấy ra.

- **Kiểm tra 3 – ngưỡng:** liveLabel phải là một trong 3 loại rác (không phải nhóm "không có rác") và liveConfidence ≥ 0,8. Không đạt thì đặt bộ đếm ổn định về 0.
- **Cập nhật bộ đếm ổn định:** cùng loại với khung trước thì cộng 1 vào số khung và cộng độ tin cậy vào tổng; đổi loại thì lưu loại mới, đặt số khung = 1 và tổng = độ tin cậy của khung này.
- **Kích hoạt:** đủ 2 khung thì gọi onDetect(loại, tổng / số khung) đúng một lần, đặt hết cooldown = bây giờ + 5 giây, đặt sẵn_sàng = false và đưa bộ đếm rời về 0.
- **Đối chiếu với Bảng 3.5:** "Chờ" ứng với sẵn sàng và bộ đếm bằng 0; "Xác nhận" ứng với bộ đếm bằng 1; "Nghỉ" ứng với đang cooldown; "Chờ vật rời khung" ứng với sẵn_sàng = false.

Đánh giá thuật toán

- **Thời gian phản hồi:** cần ít nhất 2 khung liên tiếp ở tần số 2 khung/giây, tức khoảng 1 giây kể từ khi vật vào khung (cộng thêm thời gian suy luận).
- **Khoảng cách tối thiểu giữa hai lần mở:** 5 giây cooldown cộng ít nhất 2 khung trống (khoảng 1 giây), tức khoảng 6 giây. Một vật đặt yên trước camera chỉ được tính một lần.
- **Chi phí phân loại:** 180 × 512 ≈ 92 nghìn phép nhân – cộng mỗi khung, không đáng kể so với thời gian chạy mô hình (khoảng 0,5 giây/khung bằng GPU trên máy thử TECNO CM6).
- **Khả năng chống mở nhầm:** có ba lớp bảo vệ. Ngưỡng 0,8 loại kết quả không chắc chắn; nhóm "không có rác" loại trường hợp camera nhìn vào tay, mặt người hay mặt bàn; yêu cầu 2 khung liên tiếp cùng loại loại các khung nhiễu đơn lẻ.

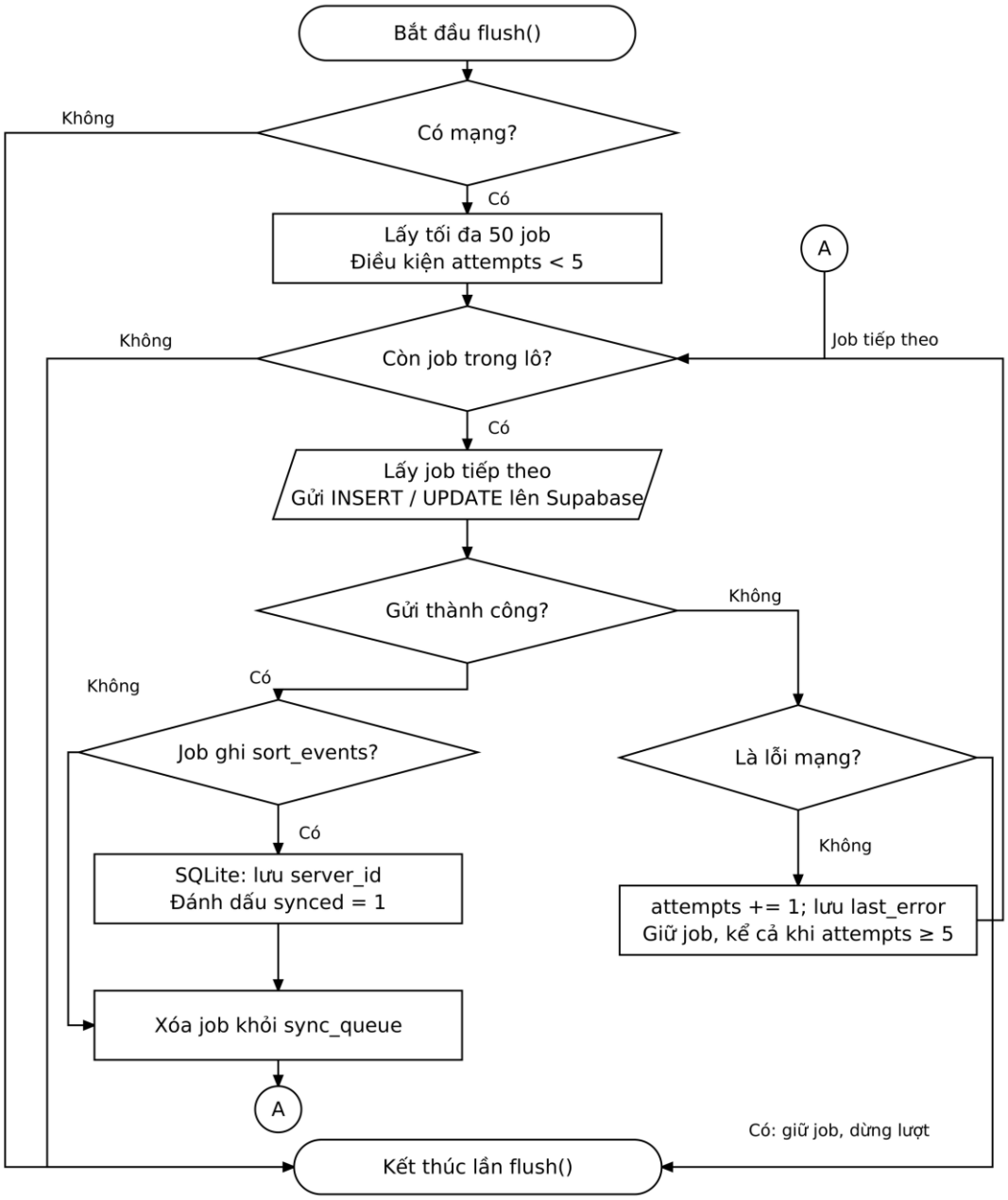
Trường hợp biên và hạn chế

- Phép kiểm tra NaN chỉ xét phần tử đầu tiên của vector đầu ra.
- Ngưỡng 0,8 cố định, không tự điều chỉnh theo điều kiện ánh sáng.
- Đã chuyển sang CPU thì không thử lại GPU trong phiên, kể cả khi lỗi GPU chỉ là tạm thời.

3.4.3. Đồng bộ ngoại tuyến

Bộ máy đồng bộ chạy khi mạng được khôi phục (NetInfo) và định kỳ 15 giây: lấy tối đa 50 thao tác còn lượt thử, gửi lần lượt; thành công thì xoá khỏi hàng đợi và đánh dấu sự

kiện đã đồng bộ; lỗi mạng thì dừng, lỗi khác thì tăng số lần thử. Lịch sử hiển thị gộp dữ liệu máy chủ với các sự kiện chưa đồng bộ trên máy.



Hình 3.5: Lưu đồ một lượt đồng bộ ngoại tuyến flush()

Phân tích thuật toán Hình 3.5. flush() (core/sync/engine.ts) hiện thực mô hình lưu rồi chuyển tiếp (store-and-forward). Mỗi thao tác ghi được lưu bền trong bảng sync_queue của SQLite và chỉ bị xoá khi máy chủ đã xác nhận nhận được.

Tham số và dữ liệu

- **Tham số:** mỗi lô tối đa 50 job; tối đa 5 lần thử (MAX_ATTEMPTS); thử lại định kỳ mỗi 15 giây; cờ running ngăn hai lượt chạy song song.

- **Bảng sync_queue:** id tự tăng, entity (sort_events hoặc collection_tasks), operation (insert hoặc update), payload dạng JSON, created_at, attempts và last_error.
- **Ba nguồn kích hoạt:** ngay sau mỗi thao tác ghi; khi NetInfo báo mạng vừa được khôi phục; và bộ hẹn giờ 15 giây.

Diễn giải từng bước

- **Bước 1 – Chống chạy chồng:** nếu running = true thì trả về ngay. Ba nguồn kích hoạt có thể trùng thời điểm; cờ này bảo đảm một job không bị gửi hai lần song song.
- **Bước 2 – Kiểm tra mạng:** NetInfo.fetch() không có kết nối thì kết thúc ngay, không tốn yêu cầu HTTP nào.
- **Bước 3 – Lấy lô:** SELECT … WHERE attempts < 5 ORDER BY id ASC LIMIT 50. Điều kiện attempts nằm ngay trong câu truy vấn, để các job đã hỏng không chiếm chỗ trong lô và chặn các job phía sau.
- **Bước 4 – Gửi:** với insert, gọi .insert(payload).select('id').single() để lấy id do máy chủ cấp; với update, tách id ra khỏi payload rồi gọi .update(phần còn lại).eq('id', id).
- **Bước 5 – Thành công:** với sort_events, SQLite lưu server_id và đặt synced = 1 cho dòng có cùng local_id; sau đó xoá job khỏi sync_queue. Điểm nối A trên lưu đồ quay lại bước "Còn job trong lô?".
- **Bước 6 – Thất bại:** lỗi được phân loại theo nội dung thông điệp ("network request failed", "failed to fetch", "network error"). Lỗi mạng thì dừng cả lượt và không tăng attempts. Lỗi khác (ví dụ bị RLS từ chối, dữ liệu sai ràng buộc) thì attempts + 1, lưu last_error và chuyển sang job tiếp theo.

Tính chất của thuật toán

- **Bảo toàn thứ tự:** job được gửi theo id tăng dần (FIFO). Với cùng một công việc, cập nhật "nhận việc" luôn được gửi trước "hoàn tất". Khi gặp lỗi mạng, thuật toán dừng hẳn thay vì bỏ qua job đó, nên không job nào phía sau vượt lên trước.
- **Tính bền:** job chỉ bị xoá sau khi máy chủ trả về thành công. Ứng dụng bị tắt giữa chừng thì job vẫn còn trong SQLite và được gửi ở lần chạy sau.
- **Gửi ít nhất một lần (at-least-once):** một job có thể đến máy chủ nhiều lần nếu phản hồi bị mất. Vì vậy thao tác insert cần khoá chống trùng local_id. Thao tác update gán giá trị cố định, nên gửi lại vẫn cho cùng kết quả.

- **Cách ly lỗi:** một job lỗi dữ liệu chỉ làm tăng attempts của chính nó, không chặn các job khác. Đủ 5 lần thì job không được lấy ra nữa nhưng vẫn được giữ lại cùng last_error để kiểm tra.
- **Độ phức tạp:** một lượt flush gửi tối đa 50 yêu cầu tuần tự, tức O(n) với n là số job trong lô. Cách gửi tuần tự đơn giản và giữ được thứ tự, đổi lại sẽ chậm khi tồn nhiều job sau thời gian dài mất mạng (cần nhiều lượt flush).
- **Xung đột:** giải quyết theo last-write-wins dựa trên thời điểm tạo. Cách này phù hợp vì sự kiện bỏ rác chỉ ghi thêm và hiếm khi bị sửa đồng thời.

Trường hợp biên và hạn chế

- **Trùng local_id khi gửi lại:** nếu máy chủ đã lưu sự kiện nhưng phản hồi bị mất do rớt mạng, lần gửi lại sẽ vi phạm ràng buộc unique và bị tính là lỗi thường. Sau 5 lần, sự kiện đã có trên máy chủ nhưng trong máy vẫn ở synced = 0. Hướng khắc phục: coi lỗi trùng local_id là đã đồng bộ thành công.
- **Update không có dòng nào bị sửa:** thao tác update không yêu cầu máy chủ trả lại các dòng đã sửa. Nếu RLS không cho sửa dòng đó, Supabase vẫn trả về thành công với 0 dòng bị ảnh hưởng, và job bị xoá như đã gửi được. Hướng khắc phục: thêm .select() để kiểm tra số dòng đã cập nhật.

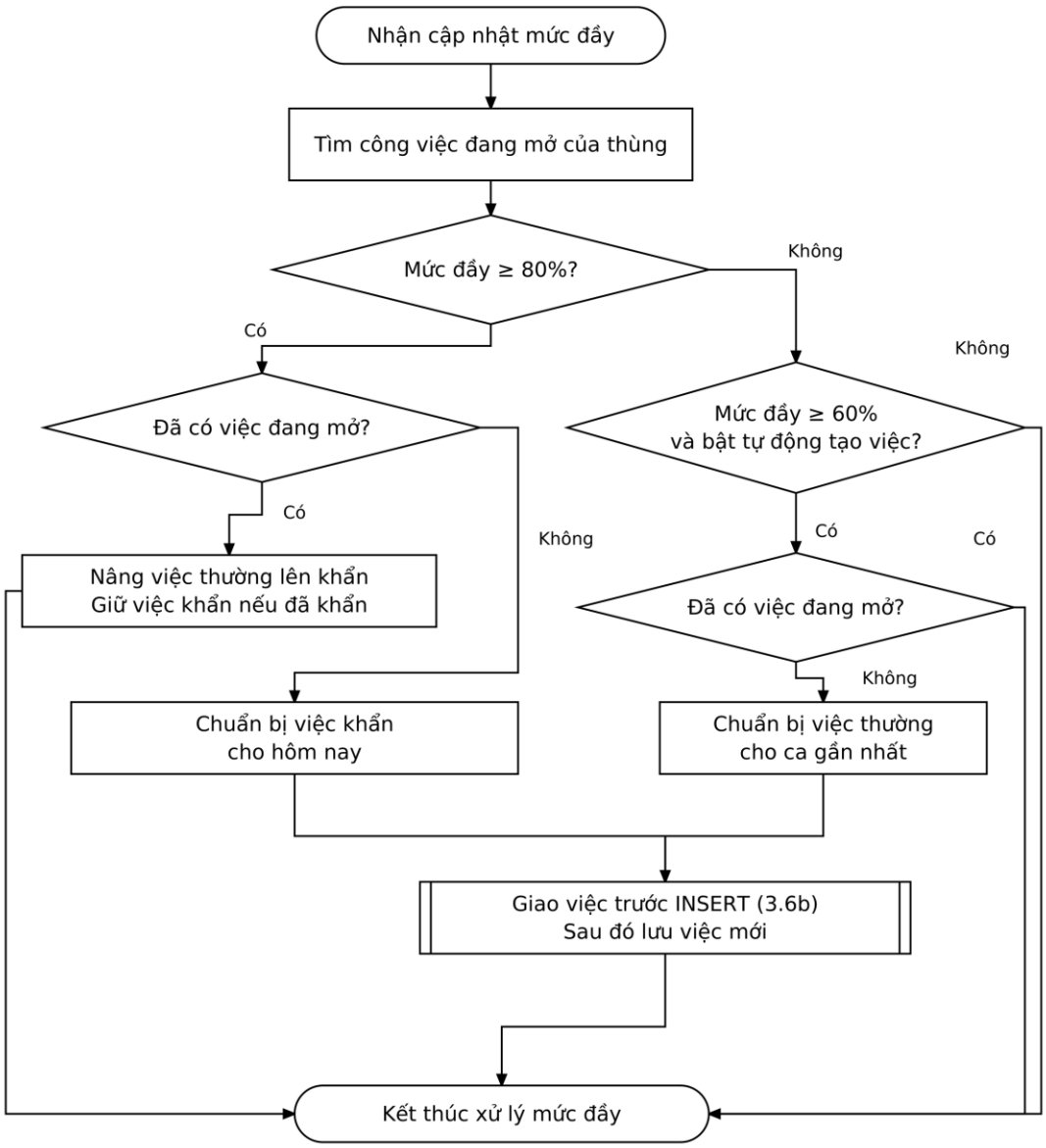
3.4.4. Tự động hoá phía máy chủ

Bảng 3.6: Các quy tắc tự động hoá

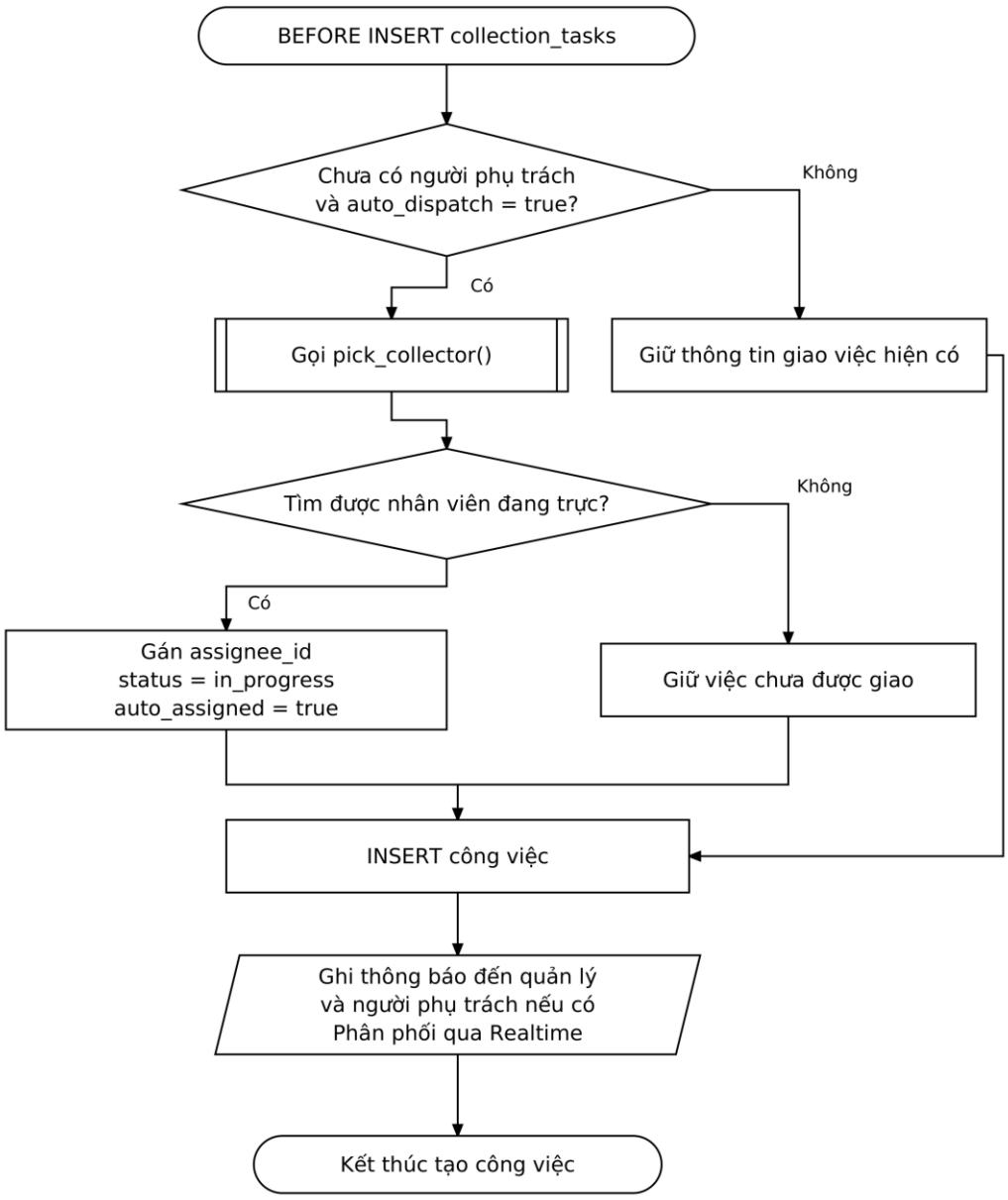
Sự kiện	Xử lý tự động (trigger / hàm)
Ngăn vượt 60%	create_task_when_full → auto_create_task: tạo việc "lịch ca" cho ca gần nhất (nếu chưa có việc mở)
Ngăn vượt 80%	Tạo việc khẩn trong ngày, hoặc nâng việc đang có lên khẩn; báo hộ gia đình nếu là thùng nhà họ
Việc mới chưa có người	auto_dispatch_task (BEFORE INSERT) → pick_collector: nhân viên đang trực, ít việc trong ngày nhất
Việc được tạo / giao / hoàn tất / chấm điểm	notify_task_created, notify_task_updated → ghi bảng notifications
Hoàn tất việc	empty_bins_on_done: thùng về 0%; reward_sorting_quality: +5 điểm nếu phân loại đúng

Mỗi 5 phút (pg_cron)	run_automation(): (a) thùng không có nhịp tim quá ngưỡng → offline; (b) việc chưa ai nhận quá hạn → tự giao, việc khẩn chậm → nhắc; (c) dự báo theo tốc độ 24 giờ, đầy trong 12 giờ → lên lịch

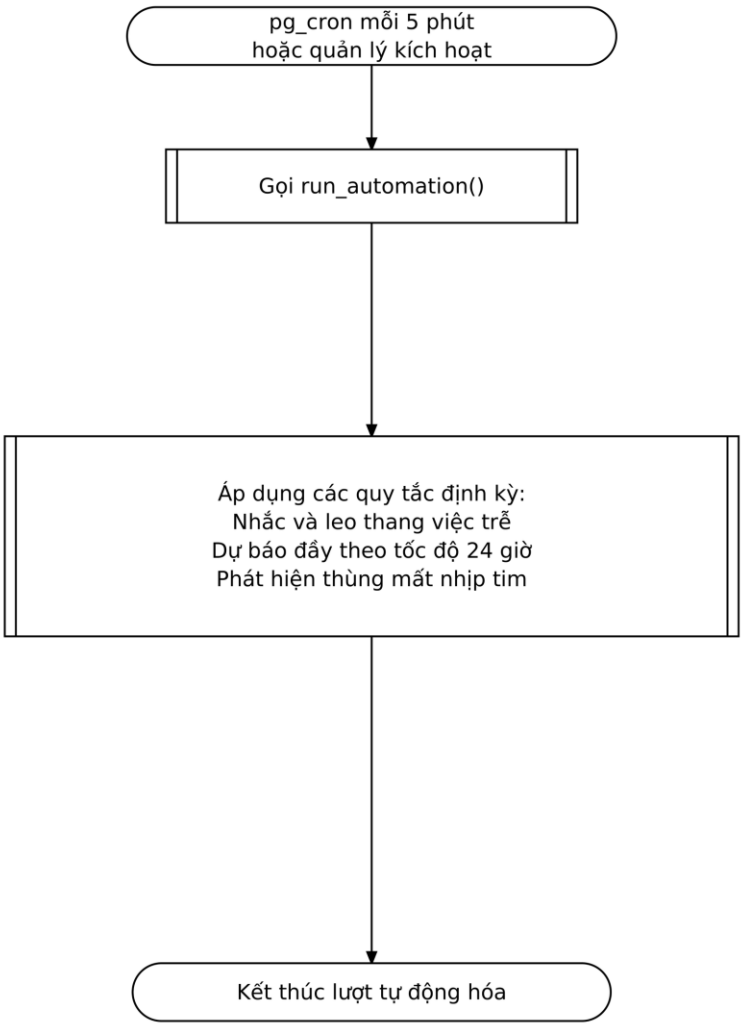
Ứng dụng lắng nghe bảng notifications qua Realtime; khi có dòng mới, hiển thị thông báo hệ thống (expo-notifications) và tải lại dữ liệu liên quan. Kiosk và app hộ gia đình gửi nhịp tim (hàm device_heartbeat) mỗi phút khi đang kết nối thùng.



Hình 3.6(a): Lưu đồ tạo và nâng ưu tiên công việc theo mức đầy



Hình 3.6(b): Lưu đồ tự động giao việc trước khi lưu công việc



Hình 3.6(c): Lưu đồ các quy tắc tự động hoá định kỳ

Phân tích thuật toán Hình 3.6. Ba lưu đồ ứng với ba thành phần PostgreSQL chạy trên máy chủ: trigger create_task_when_full (a), trigger auto_dispatch_task (b) và hàm run_automation() (c). Đặt ở máy chủ nên quy tắc được áp dụng thống nhất cho mọi ứng dụng, kể cả kiosk không đăng nhập, và chạy trong cùng giao dịch với dữ liệu đã kích hoạt nó.

Hình 3.6(a) – Tạo và nâng ưu tiên việc theo mức đầy

- **Kích hoạt theo ngưỡng:** trigger AFTER UPDATE trên bảng bins chạy cho từng ngăn. Điều kiện là "vượt ngưỡng": mức đầy cũ < ngưỡng và mức đầy mới ≥ ngưỡng. Vì vậy trigger chỉ phản ứng một lần khi ngăn đi qua mốc 60% hoặc 80%, không phản ứng lại ở mỗi lượt bỏ rác sau đó.

- **Số lượt tương ứng:** mỗi lượt bỏ rác tăng 2%, nên một ngăn rỗng vượt 60% ở lượt thứ 30 và vượt 80% ở lượt thứ 40.
- **Nhánh ≥ 80% (xét trước):** tìm việc chưa hoàn tất của thùng. Chưa có thì gọi auto_create_task với mức khẩn, ca cả ngày hôm nay. Có việc thường thì nâng lên khẩn, chuyển sang ca cả ngày, ngày thực hiện = min(ngày đã lên lịch, hôm nay), và ghi chú "đầy nhanh hơn dự báo". Việc đã khẩn thì giữ nguyên. Nếu thùng thuộc một hộ gia đình, hệ thống còn báo cho chủ hộ "Thùng nhà bạn sắp đầy".
- **Nhánh ≥ 60%:** chỉ chạy khi quy tắc lên lịch dự đoán (predictive_schedule) được bật. auto_create_task tạo việc thường cho ca kế tiếp (lấy từ next_shift()). Hàm tự kiểm tra lại việc đang mở trước khi chèn, nên không bao giờ tạo hai việc mở cho cùng một thùng.

Hình 3.6(b) – Tự động giao việc

- **BEFORE INSERT:** trigger sửa trực tiếp dòng sắp được lưu, nên không cần thêm lệnh UPDATE thứ hai, và trigger thông báo AFTER INSERT thấy ngay người phụ trách.
- **Chiến lược chọn người (pick_collector):** lọc nhân viên có on_duty = true, rồi sắp xếp theo (1) số việc chưa xong trong ngày của việc mới, (2) tổng số việc chưa xong, (3) ngẫu nhiên khi các tiêu chí trên bằng nhau, và lấy người đầu tiên. Đây là chiến lược tham lam chọn người ít việc nhất (least-loaded), giúp chia đều việc giữa các nhân viên.
- **Không có ai đang trực:** pick_collector trả về rỗng, việc được lưu ở trạng thái pending và quản lý nhận thông báo để giao tay.
- **Chi phí:** với mỗi nhân viên đang trực cần hai truy vấn đếm, nên chi phí tăng theo số nhân viên × số việc. Với quy mô một khu vực (vài chục nhân viên, vài trăm việc) thời gian thực hiện là tức thời.

Hình 3.6(c) – Quy tắc định kỳ

- **Lịch chạy:** pg_cron gọi run_automation() mỗi 5 phút (biểu thức */5 * * * *). Khi quản lý bấm chạy tay, hàm kiểm tra người gọi phải có vai trò admin.
- **Quy tắc 1 – Mất kết nối:** thùng có last_seen_at cũ hơn offline_after_min (mặc định 10 phút) được đặt is_online = false và quản lý nhận thông báo. Câu UPDATE … RETURNING vừa sửa vừa lấy danh sách, nên chỉ báo những thùng vừa chuyển trạng thái, không báo lặp lại.
- **Quy tắc 2 – Leo thang:** xét các việc chưa xong, chưa từng leo thang, đã tạo quá escalate_after_min (mặc định 30 phút) và đang chưa có người hoặc ở mức khẩn. Việc

chưa có người thì thử pick_collector một lần nữa; vẫn không có ai thì báo quản lý. Việc khẩn đã có người thì nhắc nhân viên đó và báo quản lý. Cột escalated_at bảo đảm mỗi việc chỉ bị leo thang một lần.

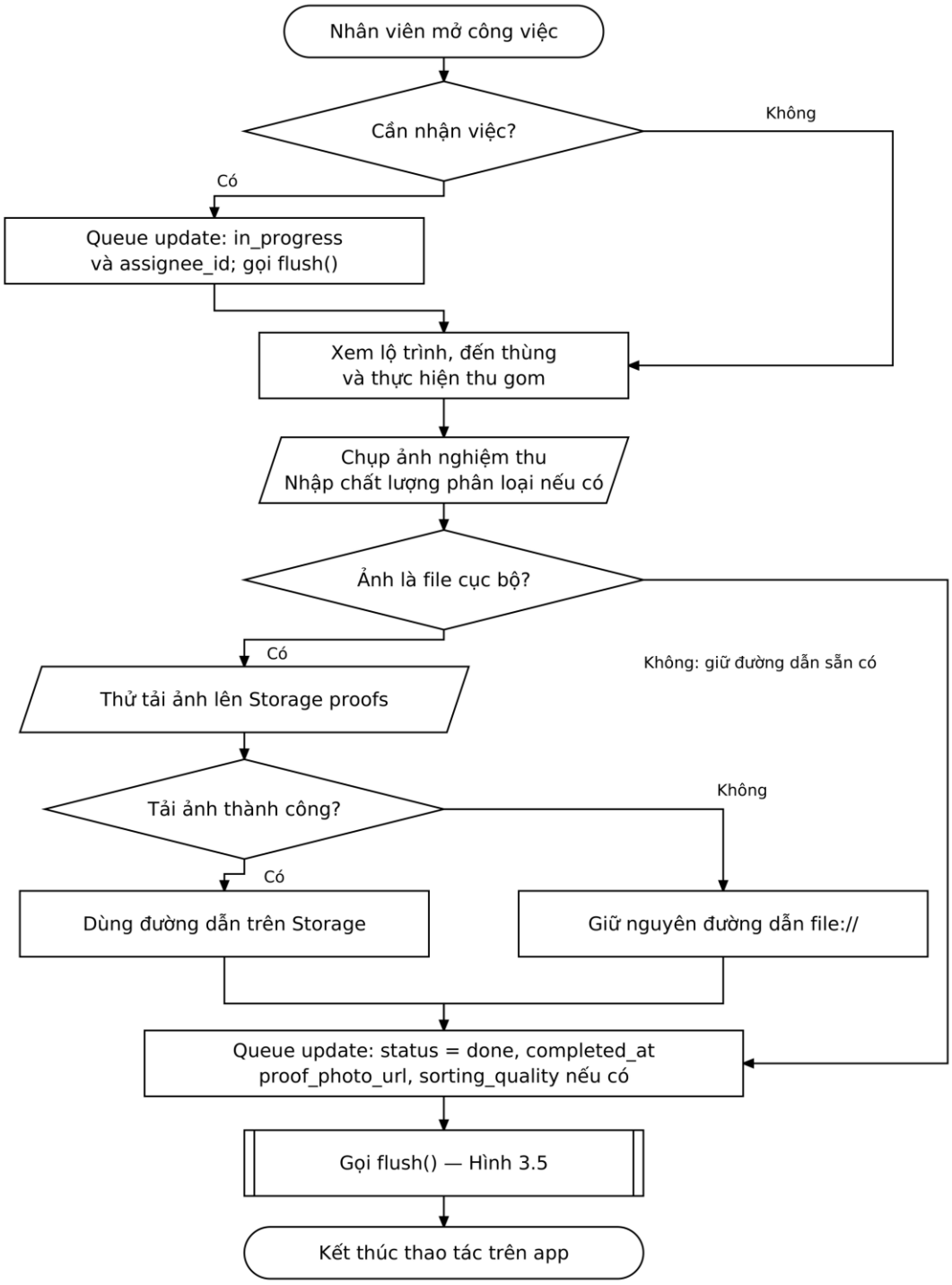
- **Quy tắc 3 – Dự báo:** với mỗi thùng chưa có việc mở, xét từng ngăn dưới 80%. Tốc độ đầy = số lượt trong 24 giờ × 2% / 24 giờ; thời gian còn lại = (80% − mức đầy) / tốc độ. Nếu còn ≤ 12 giờ thì tạo việc thường, và mỗi thùng chỉ tạo một việc.
- **Quy tắc 4 – Dọn dữ liệu:** xoá thông báo cũ hơn 30 ngày.
- **Kết quả trả về:** JSON gồm số thùng mất kết nối, số việc leo thang và số việc được lên lịch, hiển thị cho quản lý ở tab Tự động.

Đánh giá thuật toán

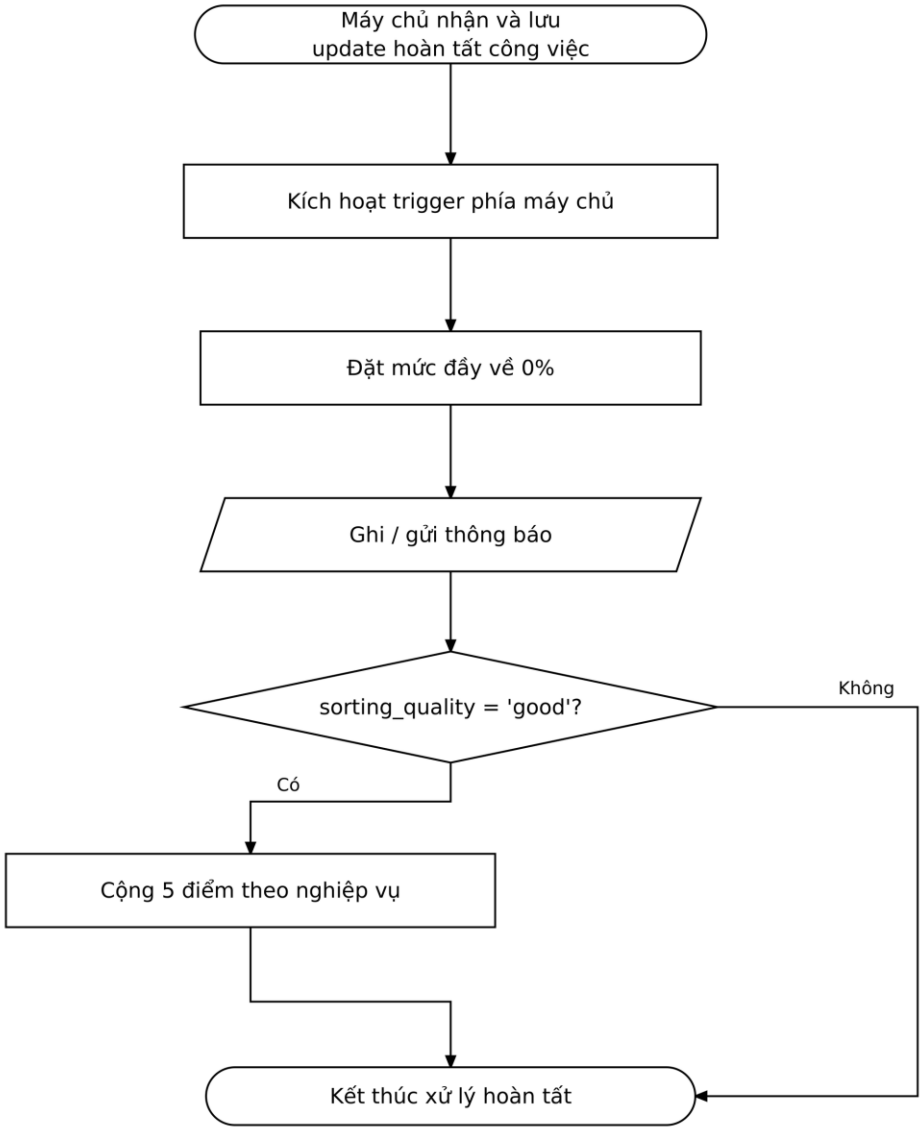
- **Không tạo trùng, không báo lặp:** có ba cơ chế cùng bảo đảm điều này: điều kiện vượt ngưỡng (chỉ kích hoạt một lần), kiểm tra việc đang mở trước khi tạo, và cột escalated_at.
- **Vòng đời thùng khép kín:** khi việc hoàn tất, mức đầy về 0% (Hình 3.7(b)). Lần vượt ngưỡng tiếp theo lại kích hoạt trigger từ đầu.
- **Hạn chế:** dự báo là tuyến tính và chỉ dựa trên số lượt bỏ rác với mức cố định 2% mỗi lượt, chưa dùng số đo thật từ cảm biến. Ngưỡng 60% / 80% và 12 giờ được đặt cố định trong mã SQL.

3.4.5. Lộ trình thu gom

Đầu vào: vị trí GPS của nhân viên (nếu gần khu vực) và toạ độ các thùng đang phụ trách. Thuật toán: láng giềng gần nhất rồi cải thiện 2-opt; nếu không có GPS thì thử xuất phát từ từng thùng và chọn lộ trình ngắn nhất. Kết quả hiển thị trên bản đồ Leaflet (WebView) và mở Google Maps để chỉ đường.



Hình 3.7(a): Lưu đồ nhận và hoàn tất công việc thu gom trên ứng dụng



Hình 3.7(b): Lưu đồ xử lý hoàn tất công việc phía máy chủ

Phân tích thuật toán Hình 3.7. Thuật toán thu gom gồm hai hàm phía ứng dụng, claimTask() và completeTask() trong taskActions.ts (a), cùng các trigger cập nhật trên bảng collection_tasks phía máy chủ (b). Ranh giới giữa hai phần rõ ràng: ứng dụng kết thúc khi cập nhật đã được xếp vào hàng đợi, còn các hiệu ứng nghiệp vụ chỉ xảy ra khi máy chủ nhận được cập nhật.

Đầu vào và đầu ra

- **Đầu vào:** mã công việc, mã nhân viên, đường dẫn ảnh nghiệm thu, và chất lượng phân loại ('good' hoặc 'mixed'; chỉ có với thùng của hộ gia đình).

- **Đầu ra:** completeTask() trả về true nếu ảnh đã lên Storage, để giao diện báo cho nhân viên biết ảnh đã được lưu trên máy chủ hay mới chỉ lưu trong máy.

Diễn giải từng bước trên ứng dụng (Hình 3.7(a))

- **Nhận việc:** việc chưa có người thì xếp cập nhật status = in_progress cùng assignee_id vào hàng đợi rồi gọi flush(). RLS chỉ cho nhân viên sửa việc chưa ai nhận hoặc việc của chính mình. Việc đã được giao (thủ công hoặc tự động) thì bỏ qua bước này.
- **Thu gom:** nhân viên xem lộ trình (mục 3.4.5), đến thùng và thực hiện thu gom.
- **Ảnh nghiệm thu:** nếu đường dẫn bắt đầu bằng file: (ảnh vừa chụp), ứng dụng tải ảnh lên bucket riêng tư proofs với tên "<mã việc>-<thời điểm>.jpg". Tên có thời điểm nên chụp lại không ghi đè ảnh cũ. Nếu ảnh đã là đường dẫn trên máy chủ thì dùng luôn.
- **Tải ảnh thất bại:** lỗi được bắt lại, ứng dụng giữ nguyên đường dẫn file:// và tiếp tục hoàn tất. Mất mạng tại hiện trường không chặn việc ghi nhận công việc.
- **Hoàn tất:** xếp cập nhật status = done, completed_at, proof_photo_url và sorting_quality (nếu có) vào hàng đợi rồi gọi flush() (Hình 3.5).

Diễn giải phía máy chủ (Hình 3.7(b))

- **empty_bins_on_done:** trigger AFTER UPDATE OF status, chỉ chạy khi việc chuyển sang done lần đầu (trạng thái cũ khác done). Trigger đặt mức đầy mọi ngăn của thùng về 0.
- **reward_sorting_quality:** trigger AFTER UPDATE OF sorting_quality, chỉ chạy khi giá trị đổi sang 'good' lần đầu. Trigger cộng 5 điểm cho chủ hộ sở hữu thùng (vai trò household).
- **notify_task_updated:** ghi thông báo "Đã thu gom" cho quản lý và chủ hộ, và thông báo kết quả đánh giá phân loại cho chủ hộ.

Đánh giá thuật toán

- **Lũy đẳng (idempotent):** các trigger chỉ chạy ở "lần đầu chuyển trạng thái". Vì vậy cập nhật hoàn tất có bị gửi lại nhiều lần (Hình 3.5 gửi ít nhất một lần) cũng không cộng điểm hai lần hay gửi thông báo lặp.
- **Đúng thứ tự:** hàng đợi gửi theo thứ tự FIFO, nên cập nhật nhận việc luôn đến máy chủ trước cập nhật hoàn tất của cùng một việc.
- **Không phụ thuộc mạng tại hiện trường:** mọi thao tác ghi đi qua hàng đợi, nên nhân viên có thể hoàn tất nhiều việc liên tiếp khi mất mạng; dữ liệu được gửi khi có mạng trở lại.

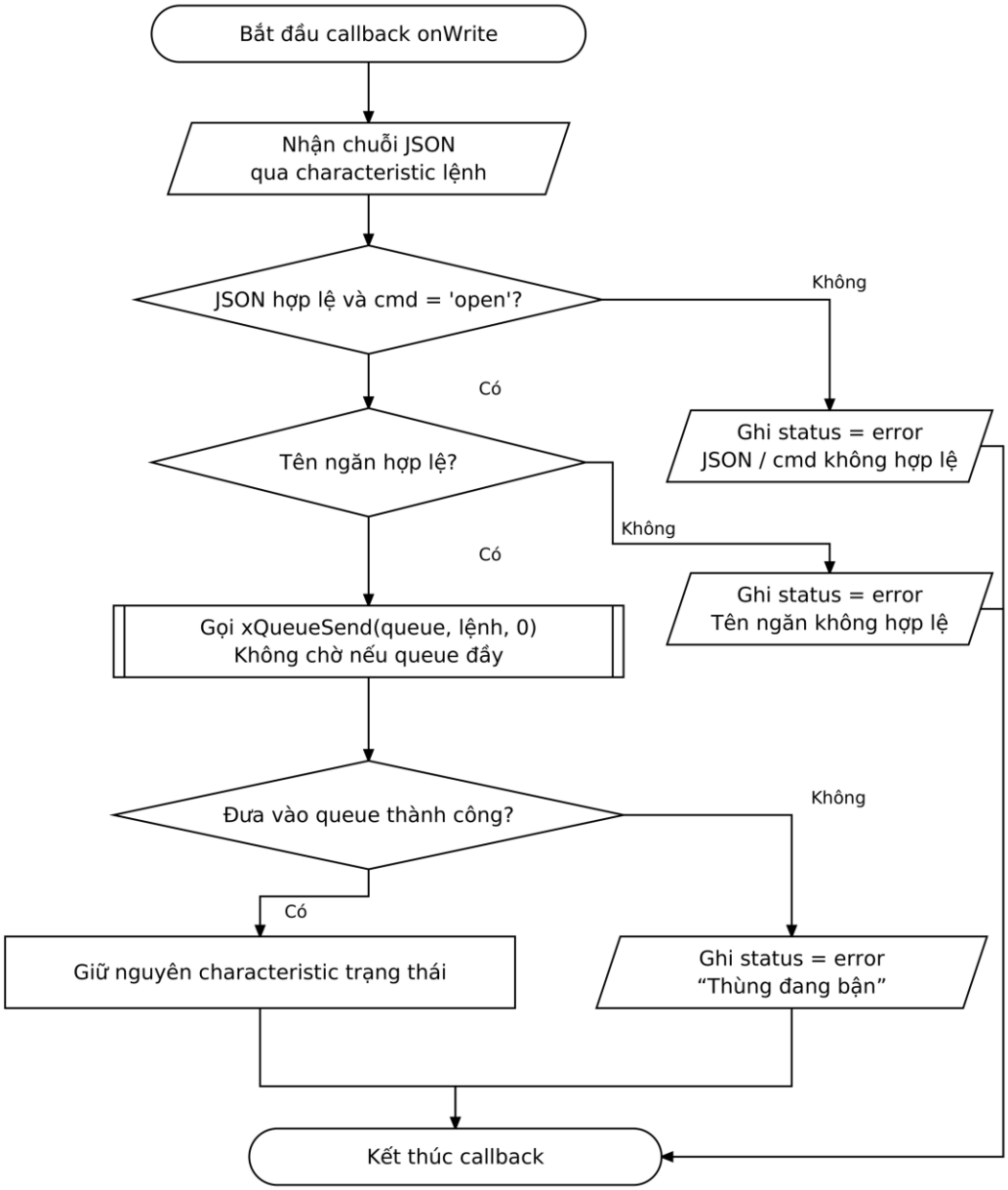
Trường hợp biên và hạn chế

- **Ảnh không được tải lại:** chưa có cơ chế tải lại ảnh. Nếu ảnh không lên được Storage, máy chủ lưu đường dẫn file://, nên chỉ máy của nhân viên đó xem được ảnh nghiệm thu.
- **Thời gian hoàn tất:** completed_at lấy theo đồng hồ điện thoại tại thời điểm bấm hoàn tất, không phải thời điểm máy chủ nhận được cập nhật.

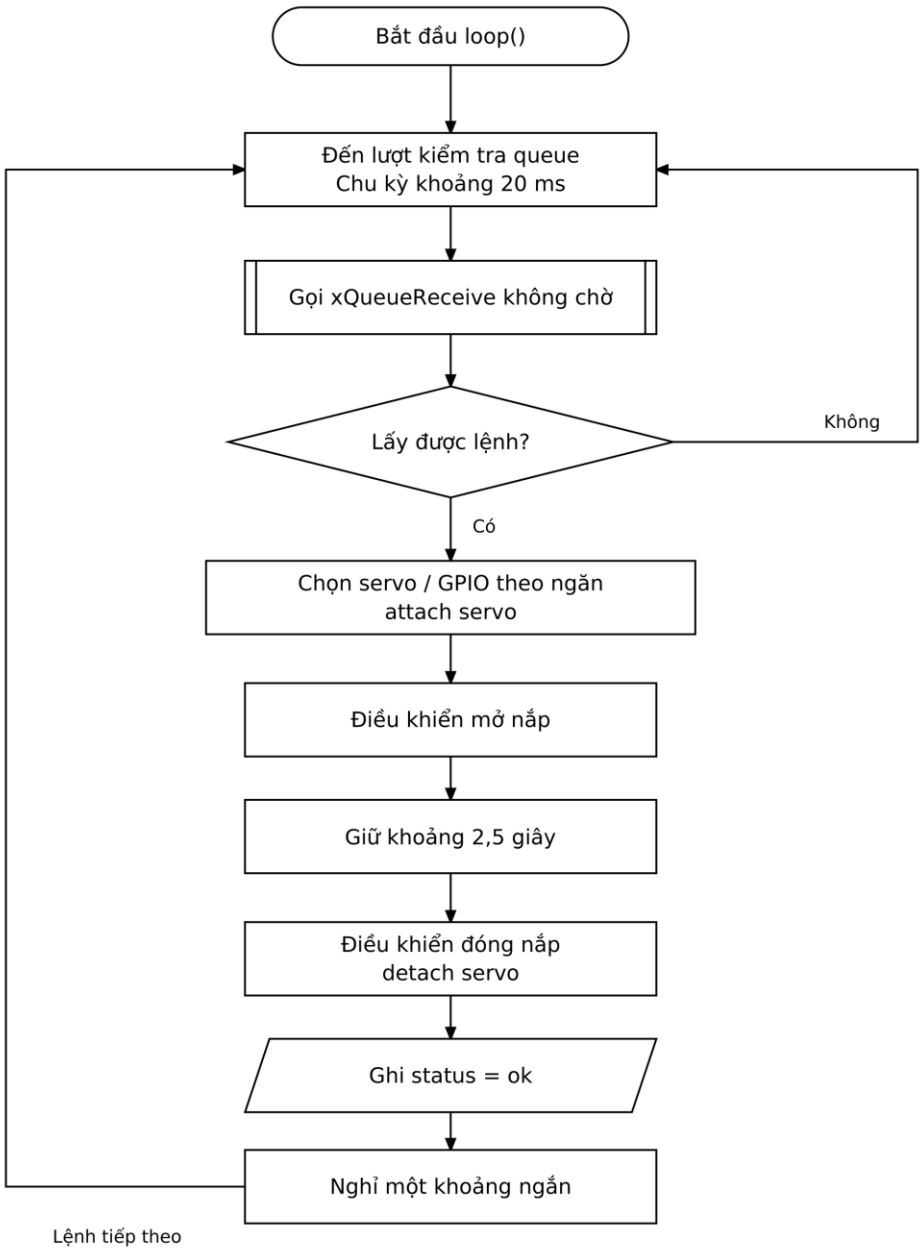
3.5. Thiết kế phần cứng và giao thức BLE

Bảng 3.7: Thiết kế phần cứng

Thành phần	**Thiết kế**
Vi điều khiển	ESP32-WROOM-32 (esp32dev), PlatformIO, thư viện ESP32Servo, ArduinoJson
Cơ cấu chấp hành	3 servo: hữu cơ GPIO 13, vô cơ GPIO 14, tái chế GPIO 27. Vô cơ, tái chế: servo 180°, mở 90° rồi đóng 0°; hữu cơ: servo quay liên tục 360°, quay thuận 300 ms để mở, quay ngược 300 ms để đóng. Cả 3 ngăn giữ nắp mở 2,5 giây
BLE	Tên "SmartBin-01"; service 6e400001-…; characteristic lệnh 6e400002-… (ghi); trạng thái 6e400003-… (đọc)
Lệnh	{"cmd":"open","bin":"huu_co"} → phản hồi {"status":"ok"} hoặc {"status":"error","message":…}
Dự phòng	MockBinController cùng giao diện BinController, bật bằng EXPO_PUBLIC_MOCK_BLE=1



Hình 3.8(a): Lưu đồ callback onWrite nhận lệnh BLE trên ESP32



Hình 3.8(b): Lưu đồ vòng lặp loop() điều khiển servo trên ESP32

Phân tích thuật toán Hình 3.8. Firmware ESP32 dùng mô hình producer – consumer: hai luồng thực thi độc lập giao tiếp qua một hàng đợi FreeRTOS. Callback onWrite (a) là bên sản xuất lệnh, loop() (b) là bên tiêu thụ lệnh và điều khiển servo.

Tham số

- **Hàng đợi:** sức chứa 5 phần tử, mỗi phần tử 1 byte là chỉ số ngăn (0 – 2) thay vì chuỗi tên ngăn, nên chép giữa hai task rất nhanh.

- **Thời gian:** chu kỳ loop() 20 ms; quay 5° mỗi 15 ms; góc mở 90°, góc đóng 0°; giữ nắp 2,5 giây; nghỉ 500 ms giữa hai lệnh.
- **Servo 360° ở ngăn hữu cơ:** tốc độ 90 ± 30, quay thuận 300 ms để mở, quay ngược 300 ms để đóng.

Diễn giải luồng tiếp nhận (Hình 3.8(a))

- **Kiểm tra lệnh:** callback onWrite chạy trong task BLE. Callback giải mã JSON bằng ArduinoJson (bộ đệm tĩnh 160 byte), kiểm tra cmd = 'open', rồi đổi tên ngăn sang chỉ số. Mỗi bước kiểm tra sai đều ghi status = error với thông điệp riêng và kết thúc callback.
- **Đưa vào hàng đợi:** xQueueSend với thời gian chờ 0, tức không chờ khi hàng đợi đầy. Đầy thì ghi status = error "Thùng đang bận"; thành công thì giữ nguyên characteristic trạng thái.
- **Không chặn BLE:** callback không gọi delay() và không điều khiển servo, vì chặn lâu trong callback làm ngăn xếp BLE hoạt động sai mà không báo lỗi.

Diễn giải luồng thực thi (Hình 3.8(b))

- **Lấy lệnh:** loop() gọi xQueueReceive không chờ. Không có lệnh thì nghỉ 20 ms rồi kiểm tra lại.
- **Servo 180° (vô cơ, tái chế):** attach, đưa về 0°, quay từ từ lên 90° (18 bước × 15 ms), giữ 2,5 giây, quay từ từ về 0°, rồi detach.
- **Servo 360° (hữu cơ):** loại servo này không giữ được góc, nên lệnh mở là quay thuận 300 ms rồi dừng, lệnh đóng là quay ngược đúng 300 ms.
- **Kết thúc lệnh:** ghi status = ok, nghỉ 500 ms, rồi xử lý lệnh tiếp theo nếu còn.

Đánh giá thuật toán

- **Thời gian một lệnh:** servo 180° mất khoảng 100 ms + 270 ms mở + 2 500 ms giữ + 270 ms đóng + 300 ms + 500 ms nghỉ ≈ 3,9 giây. Servo 360° mất khoảng 100 + 300 + 2 500 + 300 + 200 + 500 ms ≈ 3,9 giây.
- **Trường hợp hàng đợi đầy:** khi có 5 lệnh chờ, lệnh cuối phải đợi khoảng 4 × 3,9 ≈ 15,6 giây mới bắt đầu và mất khoảng 19,5 giây mới xong; lệnh thứ 6 bị từ chối. Với nhịp bỏ rác thực tế (camera cần ít nhất khoảng 6 giây giữa hai lần mở, Hình 3.4), hàng đợi gần như không bao giờ đầy.

- **An toàn đồng thời:** hàng đợi FreeRTOS tự đồng bộ giữa các task, nên không cần mutex. Chỉ có một bên tiêu thụ là loop(), nên hai nắp không bao giờ chuyển động cùng lúc; nhờ vậy dòng đỉnh không vượt khả năng của nguồn 5 V.
- **Tiết kiệm điện và giảm va đập:** servo chỉ được attach trong lúc quay và được detach sau đó. Quay từ từ 5° mỗi bước giúp giảm dòng khởi động và tiếng va đập cơ khí.

Trường hợp biên và hạn chế

- **Xác nhận chậm:** status = ok chỉ được ghi sau khi servo chạy xong, trong khi ứng dụng đọc trạng thái ngay sau khi gửi lệnh (Hình 3.3(a)). Vì vậy ứng dụng xác nhận được việc thùng đã nhận lệnh, còn việc nắp đã mở xong chưa được báo ngược về ứng dụng.
- **Servo 360° chạy theo thời gian:** góc mở của ngăn hữu cơ phụ thuộc vào tốc độ quay thực tế. Khi nguồn yếu hoặc nắp nặng, nắp có thể mở thiếu hoặc thừa; cần hiệu chỉnh hằng số SPIN_MS theo thực tế.

3.6. Thiết kế giao diện và điều hướng

Bảng 3.8: Điều hướng theo vai trò

Vai trò	**Các màn hình (tab dưới)**
Hộ gia đình	Trang chủ · Lịch sử · Thống kê · Cá nhân
Kiosk	Cấu hình kiosk → Màn chờ → Bỏ rác (camera AI bật sẵn) → Cảm ơn (tự quay về sau 5 giây)
Nhân viên	Công việc · Lộ trình · Cá nhân (+ màn Chi tiết việc)
Quản lý	Thùng rác · Điều phối · Tự động · Cá nhân

Giao diện dùng tông xanh lá – xanh ngọc (gradient) nhất quán, header bo góc, thẻ bo tròn; chuông thông báo trên header mọi màn hình chính.

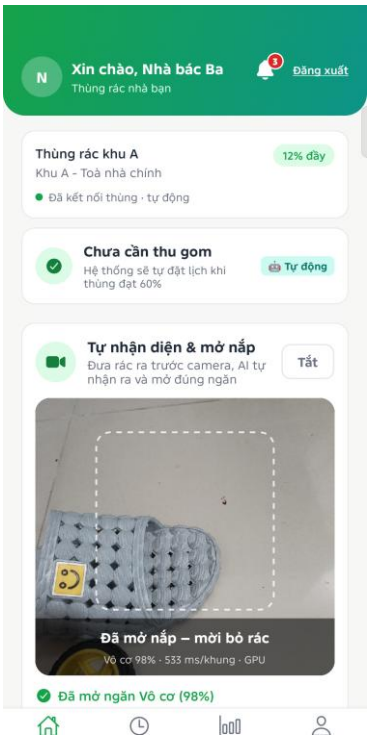
CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ

4.1. Môi trường thực hiện

Bảng 4.1: Môi trường thực hiện

Hạng mục	Chi tiết
Ứng dụng	React Native 0.76.5 (Hermes), Expo SDK 52, Expo Router 4, TypeScript
Thư viện chính	Zustand, TanStack Query, expo-sqlite, react-native-vision-camera 4, react-native-fast-tflite, react-native-ble-plx, expo-notifications, expo-location, Leaflet (WebView)
Máy chủ	Supabase (PostgreSQL, Auth, Realtime, Storage, pg_cron)
AI	MobileCLIP-S0; chuyển đổi bằng onnxslim, onnx2tf; đánh giá bằng Python
Thiết bị thử	Điện thoại TECNO CM6, Android 16, arm64; máy ảo Android
Quản lý mã nguồn	https://github.com/menvo-embedded/smartbin-hcmute

4.2. Kết quả giao diện

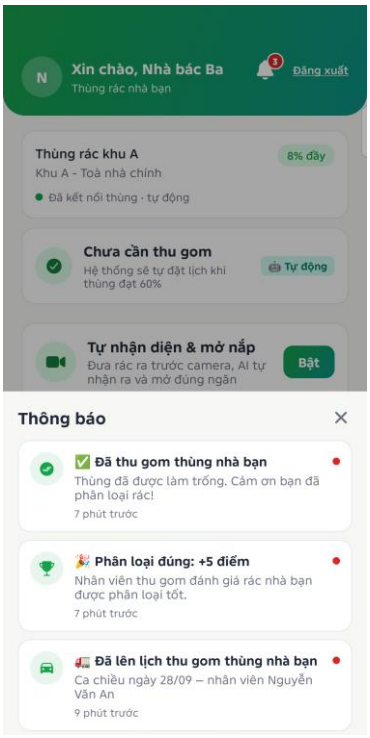


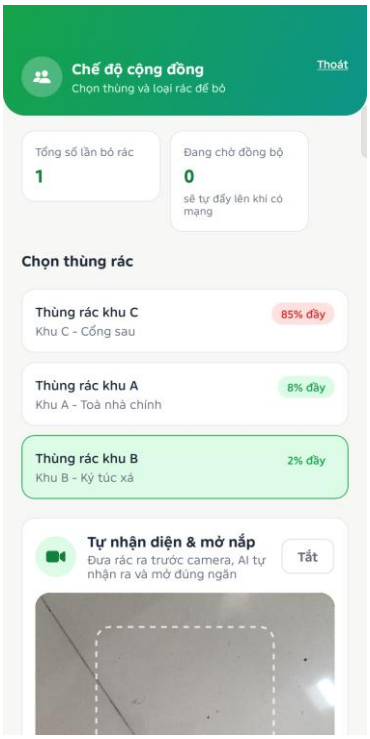


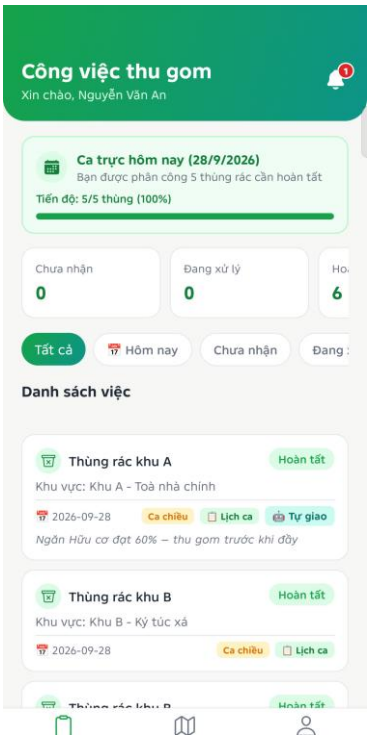


Trang chủ: camera AI tự mở nắp *Cá nhân: điểm, tác động* *Thống kê*

Hình 4.1: Giao diện Hộ gia đình

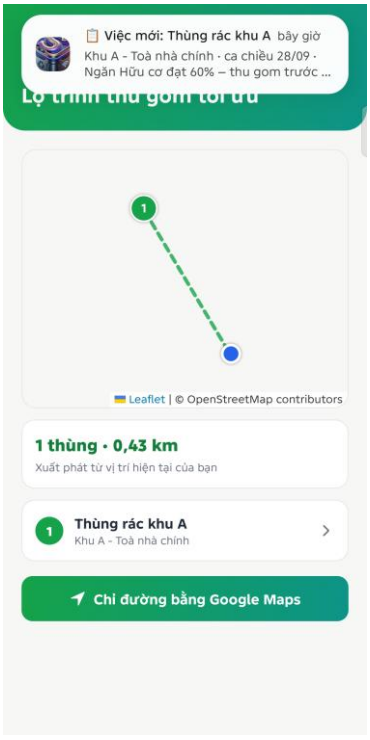


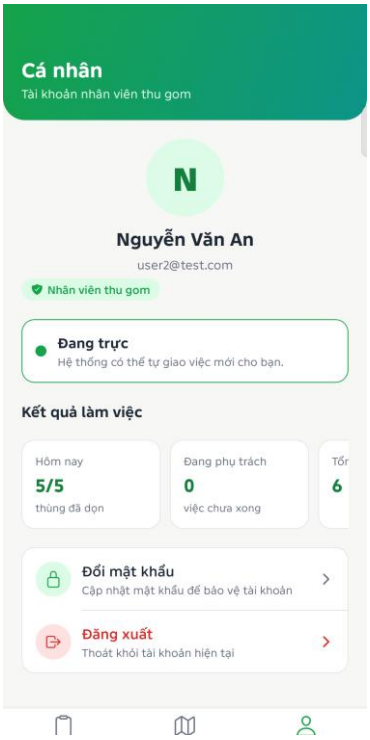


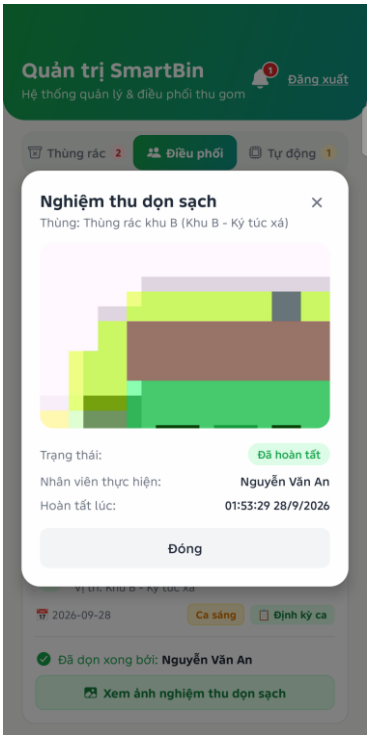


Thông báo *Kiosk cộng đồng* *Nhân viên: Công việc*

Hình 4.2: Thông báo, kiosk và danh sách việc của nhân viên

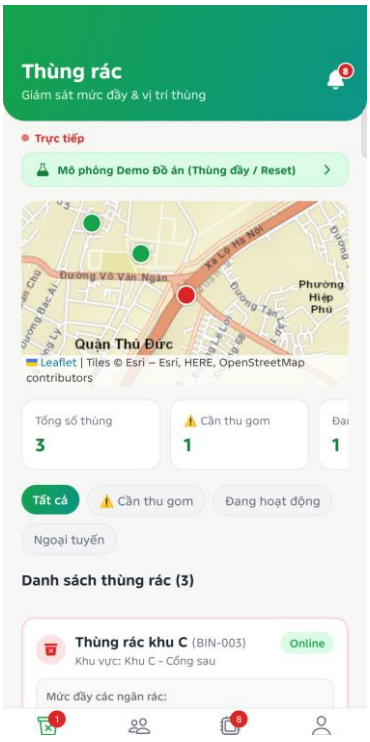


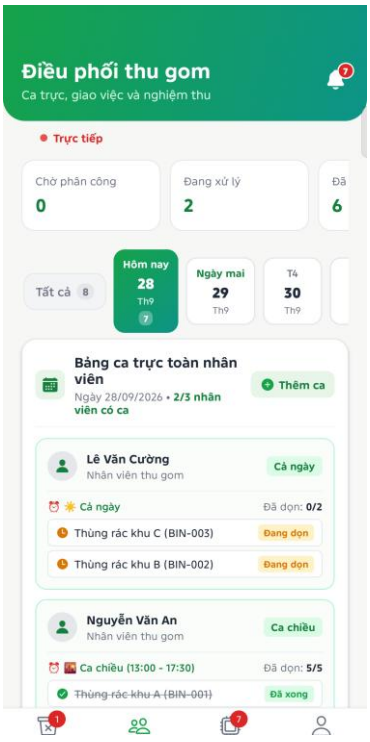


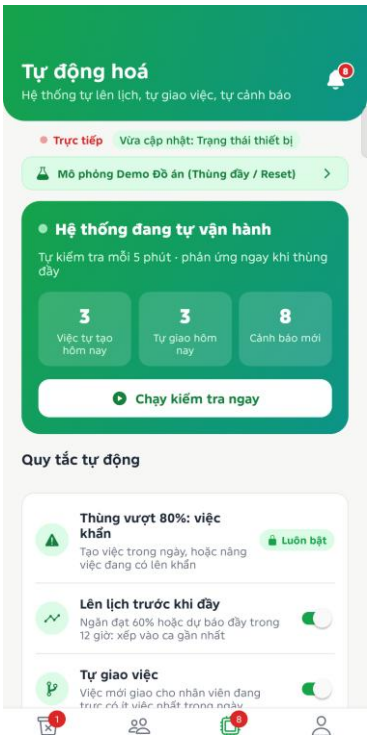


Lộ trình + thông báo *Nhân viên: Cá nhân* *Quản lý: ảnh nghiệm thu*

Hình 4.3: Nhân viên thu gom và nghiệm thu







Quản lý: Thùng rác, bản đồ *Quản lý: Điều phối* *Quản lý: Tự động hoá*

Hình 4.4: Giao diện Quản lý

4.3. Kết quả mô hình AI

Ứng dụng Android tích hợp MobileCLIP-S0 chuyển sang TensorFlow Lite, kết hợp 180 câu mô tả để phân loại zero-shot ba nhóm rác hữu cơ, vô cơ và tái chế. Toàn bộ suy luận được thực hiện trên thiết bị, không cần gửi ảnh lên máy chủ.

Tập thử: 2.749 ảnh (RealWaste [4] và nguồn bổ sung, gộp 3 nhóm). Các câu mô tả chỉ được chỉnh trên tập kiểm định; tập thử chỉ chạy một lần để báo cáo.

Bảng 4.2: So sánh các phiên bản mô hình

Phiên bản	Dung lượng	Accuracy	Balanced acc.	Độ trễ CPU (PC)	Android
ONNX FP32 (gốc)	45,5 MB	93,16%	94,63%	27 ms	—
TFLite FP32	45,6 MB	93,16%	94,63%	82 ms	Dùng mặc định

TFLite FP16	22,9 MB	91,92%	93,71%	78 ms	Ra NaN — loại
TFLite INT8 dynamic	12,1 MB	92,91%	93,86%	342 ms	Chạy được, chậm

Bảng 4.3: Ma trận nhầm lẫn TFLite FP32 trên tập thử

Thật \ Dự đoán	**Hữu cơ**	**Vô cơ**	**Tái chế**	**Recall**
Hữu cơ (326)	320	4	2	98,16%
Vô cơ (1.080)	3	1.040	37	96,30%
Tái chế (1.343)	19	123	1.201	89,43%

Nhận xét: MobileCLIP-S0 zero-shot đạt 93,16% mà không cần huấn luyện lại. Mô hình nhầm nhiều nhất là tái chế → vô cơ (123 ảnh), chủ yếu là bao bì nhựa bẩn/nhăn — chính người dùng cũng hay nhầm. Lượng tử hoá INT8 dynamic range giảm dung lượng 3,8 lần, chỉ mất 0,25 điểm phần trăm nhưng chậm hơn vì phải giải lượng tử trọng số mỗi lần chạy; bản FP16 cho kết quả NaN trên Android do tràn số.

Chế độ trực tiếp (ngưỡng 0,8, cần 2 khung liên tiếp): 86,5% khung có rác được mở ngay, trong đó 97,4% mở đúng ngăn; khung không có rác được nhận là "Không có rác" với điểm 96–100%. Trên điện thoại TECNO CM6, mỗi khung mất khoảng 490 ms với GPU và 620 ms với CPU.

4.4. Kết quả kiểm thử

4.4.1. Kiểm thử máy chủ

Tệp supabase/tests/test_logic_server.sql đóng vai từng người dùng (đặt JWT giả lập + chuyển vai trò CSDL giống PostgREST), chạy trong một giao dịch rồi hoàn tác nên không để lại dữ liệu. Kết quả: 44/44 phép kiểm tra đạt.

Bảng 4.5: Kết quả kiểm thử máy chủ

Nhóm	**Nội dung kiểm tra**	**Kết quả**
Kiosk (anon)	Ghi sự kiện ẩn danh, +2% mức đầy, nhịp tim; không ghi được sự kiện mang user_id; không đọc được việc/thông báo	6/6

Hộ gia đình	+1 điểm; không ghi thay người khác; chỉ đọc hồ sơ, việc, thông báo của mình; không sửa được cấu hình	6/6
Ngưỡng 60/80%	Tự tạo việc lịch ca, tự giao người ít việc, nâng khẩn không tạo trùng, báo đúng người	7/7
Nhân viên	Chỉ thấy việc của mình; không chạy được run_automation; hoàn tất → thùng 0%, +5 điểm, thông báo	7/7
Tắt tự giao	Việc chờ; nhân viên tự nhận; không tự báo cho chính mình	5/5
Leo thang	Việc bị bỏ quên tự giao; việc khẩn chậm được nhắc; không nhắc trùng	3/3
Thiết bị	Mất nhịp tim → offline + cảnh báo; nhịp tim lại → online	3/3
Dự báo, ca trực, quản lý	Dự báo theo tốc độ; không ai trực; chỉ một người trực; quyền quản lý; ràng buộc dữ liệu	7/7

4.4.2. Kiểm thử logic ứng dụng

Tệp scripts/test-logic.ts kiểm tra các hàm thuần: dự báo đầy (6), lộ trình và khoảng cách (6), tác động môi trường, chuỗi ngày, huy hiệu (4), tra cứu rác có/không dấu (5), trạng thái việc (1), phân loại zero-shot trên toàn bộ 180 câu mô tả (3). Kết quả: 25/25 đạt.

4.4.3. Kiểm thử trên điện thoại thật

Kiểm thử thủ công cả 4 vai trò trên bản cài release, gồm chế độ mất mạng và thông báo realtime giữa quản lý và nhân viên. Các lỗi phát hiện và đã sửa:

Bảng 4.6: Lỗi phát hiện khi kiểm thử

Lỗi	**Nguyên nhân**	**Cách sửa**
Camera tự mở nắp lặp lại	Vật nằm yên trước camera được nhận lại sau mỗi 5 giây	Thêm trạng thái "chờ vật rời khung"
Lịch sử hiện 161 lượt, máy chủ có 100	Gộp cả lượt đã đồng bộ nhưng đã bị xoá trên máy chủ	Chỉ gộp lượt chưa đồng bộ
Bản đồ trống	Máy chủ tile OSM chặn yêu cầu từ WebView không có Referer	Chuyển sang tile Esri
Màn Cảm ơn kẹt ở 0 giây	Màn tab được giữ lại, bộ đếm không đặt lại	Đặt lại khi màn được mở lại

Mô phỏng 85% không tạo việc	Chọn ngăn đầu tiên dù ngăn đó đã đầy sẵn	Chọn ngăn đang dưới ngưỡng
Lỗi giao diện	Chữ tràn, bàn phím che kết quả, ký tự "→" lỗi font máy TECNO	Sửa bố cục, tự cuộn, thay ký tự

4.5. Đánh giá mức độ đạt mục tiêu

Bảng 4.7: Đánh giá mức độ đạt mục tiêu

Mục tiêu (Chương 1)	**Kết quả**
Hỗ trợ phân loại đúng 3 nhóm	Đạt: chọn tay, tra cứu, AI chụp ảnh, AI camera tự mở ngăn (93,16%)
Khuyến khích phân loại	Đạt: điểm, huy hiệu, chuỗi ngày, CO_2, đánh giá từ nhân viên (+5 điểm)
Hỗ trợ nhân viên thu gom	Đạt: việc, lộ trình tối ưu, nghiệm thu ảnh, chấm phân loại
Giám sát, điều phối thời gian thực	Đạt: bản đồ, dự báo, điều phối, lịch ca, Realtime
Tự động hoá vận hành	Đạt: tự lên lịch, tự giao việc, nhắc việc, phát hiện mất kết nối, thông báo
Hoạt động khi mất mạng	Đạt: SQLite + hàng đợi đồng bộ, đã kiểm thử tắt/bật mạng
Mô hình thùng ESP32	Đạt: firmware BLE điều khiển 3 servo; app có giả lập demo

CHƯƠNG 5: KẾT LUẬN

5.1. Kết quả đạt được

- Xây dựng hoàn chỉnh ứng dụng Android SmartBin cho 4 nhóm người dùng, chạy trên dữ liệu thật của máy chủ Supabase.
- Tích hợp mô hình khoa học MobileCLIP-S0 (CVPR 2024) chạy trên điện thoại, phân loại zero-shot đạt 93,16%; camera tự nhận diện rác và mở đúng ngăn không cần thao tác.
- Hệ thống thu gom tự vận hành phía máy chủ: tự lên lịch trước khi thùng đầy, tự giao việc cân bằng tải, tự nhắc việc trễ, phát hiện thùng mất kết nối, thông báo thời gian thực.
- Kiến trúc local-first cho phép dùng khi mất mạng; phân quyền an toàn bằng RLS.
- Bộ kiểm thử tự động (44 kiểm thử máy chủ, 25 kiểm thử logic) và kiểm thử trên thiết bị thật.

5.2. Hạn chế

- Mô hình phần cứng thử nghiệm còn ở quy mô nhỏ; độ bền cơ khí và nguồn cấp cho nhiều servo chưa được đánh giá dài hạn.
- Mức đầy được ước lượng theo số lượt bỏ rác (2%/lượt), chưa đo bằng cảm biến.
- Thông báo chỉ hiện khi app đang mở hoặc chạy nền; chưa có máy chủ push khi app đã bị tắt hẳn.
- AI còn nhầm tái chế ↔ vô cơ với bao bì nhựa bẩn; tốc độ ~0,5 giây/khung nên phải giữ rác yên khoảng 1 giây.
- Chỉ mới phát hành cho Android; dự báo đầy dùng mô hình tuyến tính đơn giản.

5.3. Hướng phát triển

- Nâng cấp cơ cấu thùng và bảo mật ghép nối BLE; gắn cảm biến siêu âm đo mức đầy thật và gửi trạng thái qua Wi-Fi/MQTT.
- Tích hợp Firebase Cloud Messaging / Expo Push để thông báo cả khi app đã tắt.
- Lượng tử hoá INT8 hoàn toàn [5] và tinh chỉnh câu mô tả / fine-tune nhẹ để tăng tốc độ và độ chính xác nhóm tái chế.
- Tối ưu lộ trình nhiều xe (bài toán VRP) có ràng buộc tải trọng và khung giờ.
- Phát hành iOS và trang web quản trị; dự báo đầy bằng mô hình chuỗi thời gian theo ngày trong tuần.

TÀI LIỆU THAM KHẢO

[1] P. K. A. Vasu, H. Pouransari, F. Faghri, R. Vemulapalli, and O. Tuzel, "MobileCLIP: Fast image-text models through multi-modal reinforced training," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 15963–15974, doi: 10.1109/CVPR52733.2024.01511.

[2] A. Radford et al., "Learning transferable visual models from natural language supervision," in Proc. 38th Int. Conf. Mach. Learn. (ICML), vol. 139, 2021, pp. 8748–8763.

[3] M. Yang and G. Thung, "Classification of trash for recyclability status," CS229 Project Report, Stanford Univ., 2016.

[4] S. Single, S. Iranmanesh, and R. Raad, "RealWaste: A novel real-life data set for landfill waste classification using deep learning," Information, vol. 14, no. 12, Art. no. 633, 2023, doi: 10.3390/info14120633.

[5] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[6] I. Ranjbar, Y. Ventikos, and M. Arashpour, "Zero-shot and few-shot multimodal plastic waste classification with vision-language models," Waste Management, vol. 202, Art. no. 114815, 2025, doi: 10.1016/j.wasman.2025.114815.

[7] G. A. Croes, "A method for solving traveling-salesman problems," Operations Research, vol. 6, no. 6, pp. 791–812, 1958.

[8] Quốc hội Việt Nam, Luật Bảo vệ môi trường số 72/2020/QH14, 2020.

[9] Chính phủ Việt Nam, Nghị định 08/2022/NĐ-CP quy định chi tiết một số điều của Luật Bảo vệ môi trường, 2022.

[10] Meta Platforms, "React Native documentation." [Online]. Available: https://reactnative.dev/docs

[11] Expo, "Expo documentation." [Online]. Available: https://docs.expo.dev

[12] Supabase, "Supabase documentation: Database, Auth, Realtime, Storage." [Online]. Available: https://supabase.com/docs

[13] Google, "LiteRT (TensorFlow Lite) documentation." [Online]. Available: https://ai.google.dev/edge/litert

PHỤ LỤC: TÀI LIỆU BÀN GIAO

Bảng PL.1: Danh mục tài liệu bàn giao

Hạng mục	**Vị trí / liên kết**
Video demo tổng hợp	https://www.youtube.com/watch?v=mfrva-ueWmg&feature=youtu.be
Mã nguồn ứng dụng và firmware	https://github.com/menvo-embedded/smartbin-hcmute
Mô hình MobileCLIP-S0 và mã đánh giá	Thư mục ai-mobileclip và src/ml trong workspace
Ứng dụng cài đặt	SmartBin-demo.apk

Toàn bộ chức năng chính đã được hoàn thành và đối chiếu với mã nguồn tại thời điểm chốt báo cáo. Video demo được chia thành các đoạn ngắn theo từng nhóm tính năng để thuận tiện chấm và đối chiếu.